



第三章 组合逻辑电路

- 一、组合电路的定义和分析
- 二、组合电路的设计
- 三、中规模集成（**MSI**）组合电路
- 四、组合逻辑电路中的竞争与冒险



组合逻辑电路的定义

逻辑电路

组合逻辑电路

现时的输出仅取决于现时的输入

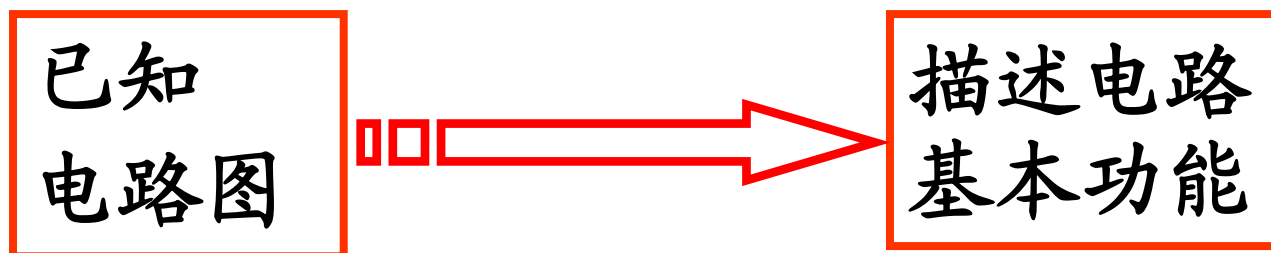
时序逻辑电路

除与现时输入有关外还与电路的原状态有关



一、组合电路的分析

基本思想:





分析方法与步骤



1. 由给定的逻辑图写出逻辑关系表达式。
2. 对逻辑表达式进行必要的化简，列出输入输出真值表。
3. 概括出电路功能的结论。



例：分析图示组合逻辑电路的逻辑功能。

解：

1. 逐级推导

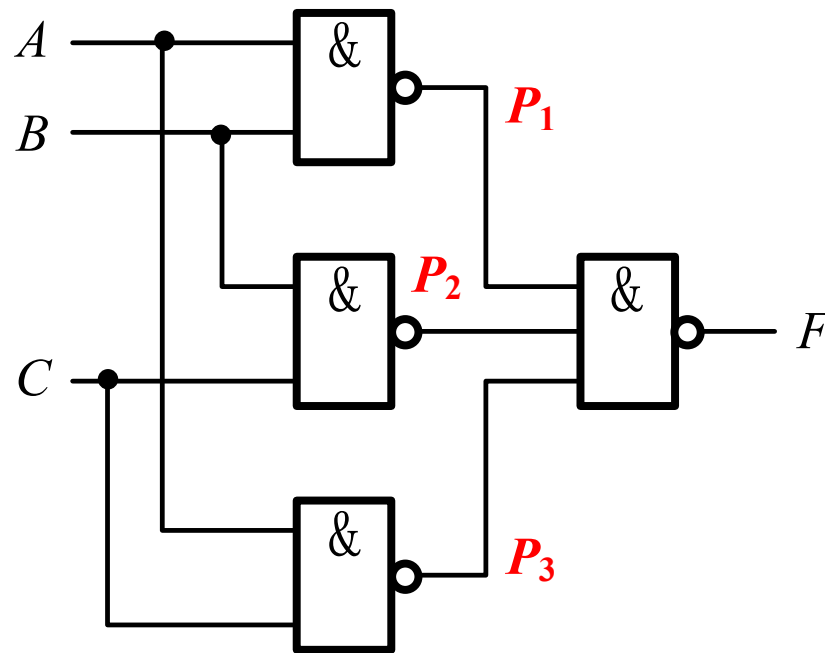
$$P_1 = \overline{AB},$$

$$P_2 = \overline{BC},$$

$$P_3 = \overline{AC}$$

2. 写出逻辑表达式

$$F = \overline{P_1 \cdot P_2 \cdot P_3} = \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}} = AB + BC + AC$$





$$F = AB + BC + AC$$

3. 列出真值表

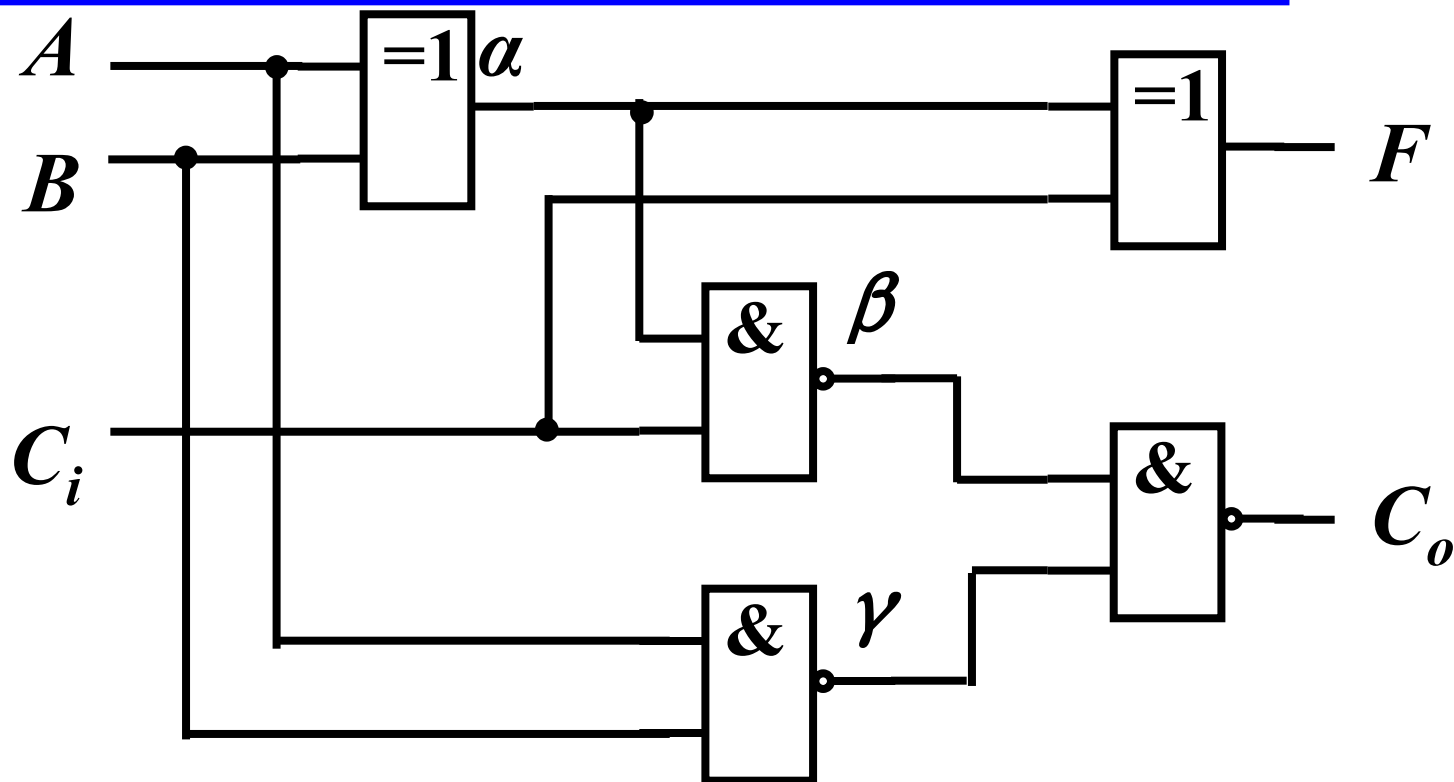
<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

4. 分析功能

它表示了一种“少数服从多数”的逻辑关系。因此可以将该电路概括为：三变量多数表决器。



例2: 分析下图的逻辑功能。



$$\alpha = A \oplus B \quad \beta = \overline{\alpha \cdot C_i} = \overline{(A \oplus B) \cdot C_i} \quad \gamma = \overline{A \cdot B}$$



代入整理后，两输出为：

$$F = \alpha \oplus C_i = A \oplus B \oplus C_i$$

$$C_o = \overline{\beta \cdot \gamma} = \overline{(A \oplus B)C_i AB}$$
$$= (A \oplus B)C_i + AB$$

功能： F 为 A 、 B 、 C_i 之和，
 C_o 为三个数之和产生的进位

真值表

A	B	C_i	F	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

一位全加器



关于加法:

举例: $A=1101$, $B=1001$, 计算 $A+B$

$$\begin{array}{r} 1101 \\ + 1001 \\ \hline 10110 \end{array}$$

(1) 逢二进一。

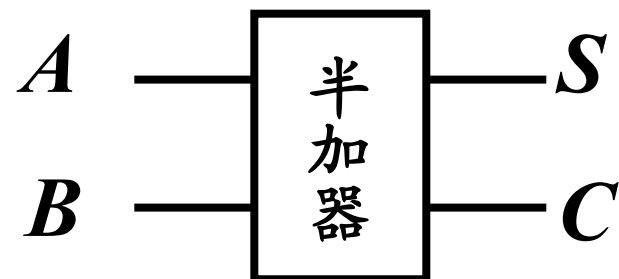
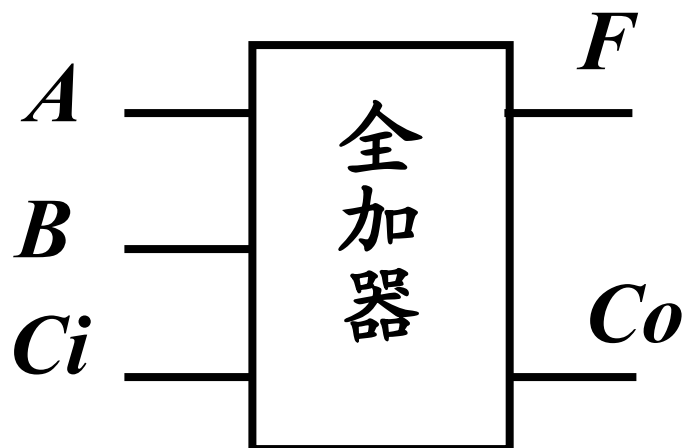
(2) 最低位是两个数最低位的相加, 不需考虑进位。

(3) 其余各位都是三个数相加, 包括加数、被、加数和低位来的进位。

(4) 任何位相加都产生两个结果: 本位和、向高位的进位。



一位集成半加器与全加器



半加运算不考虑
从低位来的进位



(1) 半加器：半加运算不考虑从低位来的进位

A ---加数； B ---被加数； S ---本位和；
 C ---进位。

真值表

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

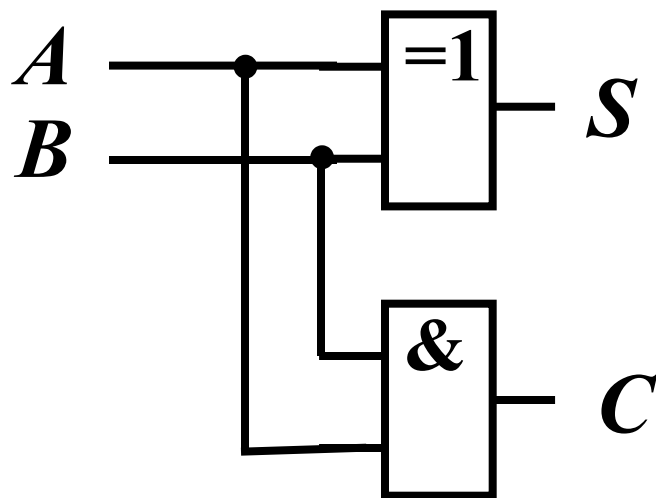
逻辑函数

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

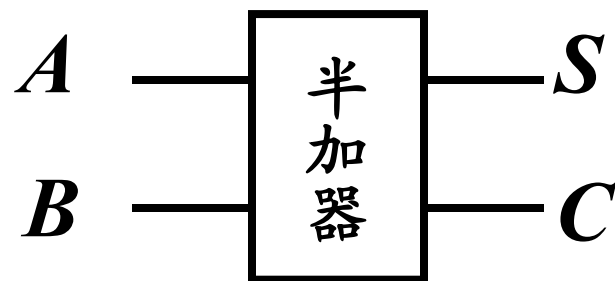
$$C = AB$$



逻辑图



中规模集成 逻辑符号



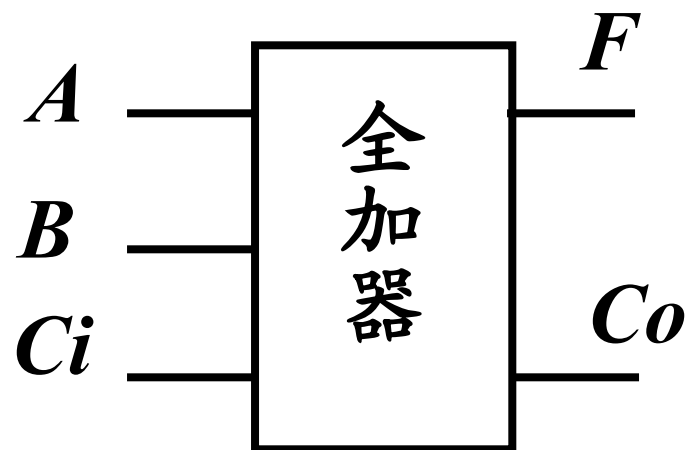


(2) 全加器:

相加过程中，既考虑加数、被加数又考虑低位的进位。集成逻辑符号：

$$F = \alpha \oplus C_i = A \oplus B \oplus C_i$$

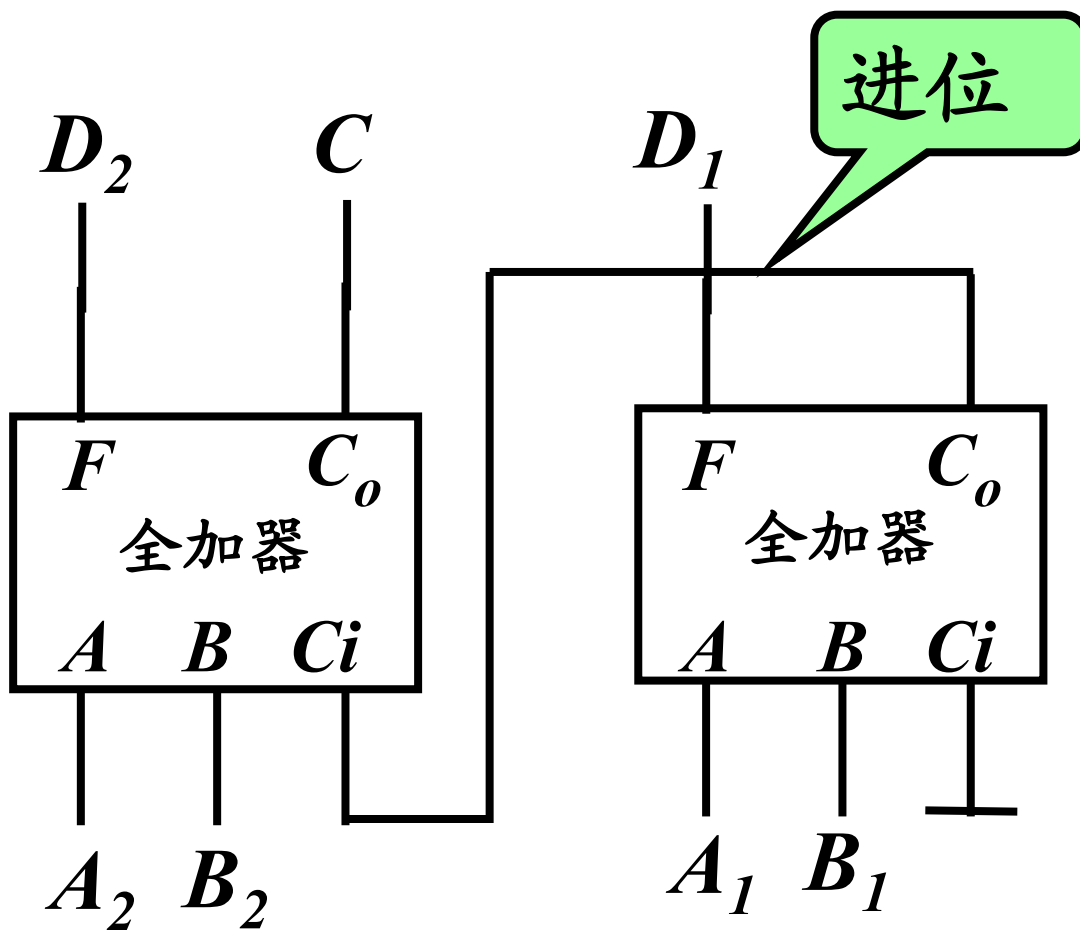
$$C_o = \overline{\beta \cdot \gamma} = \overline{(A \oplus B)C_i AB}$$
$$= (A \oplus B)C_i + AB$$





应用举例：用一位全加器构成两位加法器。

$$\begin{array}{r} A_2 \ A_1 \\ + \ B_2 \ B_1 \\ \hline C \ D_2 \ D_1 \end{array}$$





其它加法器芯片：

SN74H83---四位串行进位全加器。

SN74283---四位超前进位全加器。



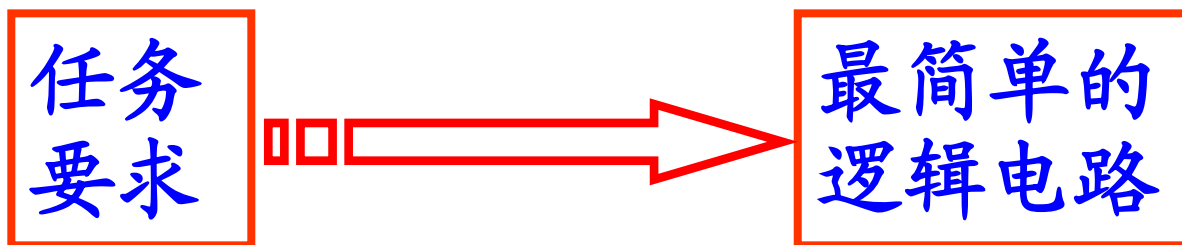
组合逻辑电路分析小结

1. 电路→逻辑表达式;
2. 化简, 列出真值表;
3. 概括结论。



二、组合电路的设计

基本思想:





二、组合电路的设计

最佳设计主要考虑问题有：

- ① 所用的逻辑器件数目最少，器件的种类最少，且器件之间的连线最简单。这样的电路称“最小化”电路。
- ② 满足速度要求，应使级数尽量少，以减少门电路的延迟。
- ③ 功耗小，工作稳定可靠。



设计步骤:

1. 逻辑抽象:

指定实际问题的逻辑符号与含义，列出真值表。

2. 选择器件类型:

根据任务和现有条件选择采用合适的器件。

3. 写出逻辑表达式:

用逻辑代数或卡诺图对逻辑表达式进行化简，并变换成与器件对应的最简表达式。

4. 画出逻辑电路图。



例1 设计一个一位二进制全减器。

① 逻辑抽象。

全减器有三个输入变量：被减数 A_n 、减数 B_n 、低位向本位的借位 C_n ；有两个输出变量：本位差 D_n 、本位向高位的借位 C_{n+1} 。

② 列真值表。

A_n	B_n	C_n	C_{n+1}	D_n
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



② 选器件。

选用异或门、与或非门分别实现。

③ 写逻辑函数式。

首先画出 C_{n+1} 和 D_n 的K图，然后根据选用的三种器件将 C_{n+1} 、 D_n 分别化简为相应的函数式。

A_n	B_n	C_n	C_{n+1}	D_n
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$A_n B_n$					
		00	01	11	10
C_n	0	0	1	0	0
	1	1	1	1	0

 C_{n+1}

$A_n B_n$					
		00	01	11	10
C_n	0	0	1	0	1
	1	1	0	1	0

 D_n

$A_n B_n$		00	01	11	10
C_n	0	0	1	0	0
	1	1	1	1	0

C_{n+1}

$A_n B_n$		00	01	11	10
C_n	0	0	1	0	1
	1	1	0	1	0

D_n

1.当用“与或非”门实现电路时，
写出相应的函数式为：

$$D_n = \overline{\overline{A_n} \overline{B_n} \overline{C_n} + \overline{A_n} B_n C_n + A_n B_n \overline{C_n} + A_n \overline{B_n} C_n}$$

$$C_{n+1} = \overline{\overline{B_n} \overline{C_n} + A_n \overline{C_n} + A_n \overline{B_n}}$$

2.当用异或门实现电路时，写出相
应的函数式为：

$$D_n = A_n \oplus B_n \oplus C_n$$

$$C_{n+1} = \overline{A_n} \overline{B_n} C_n + \overline{A_n} B_n \overline{C_n} + B_n C_n$$

$$= \overline{A_n} (B_n \oplus C_n) + B_n C_n$$

$$= \overline{\overline{A_n} (B_n \oplus C_n)} \cdot \overline{B_n C_n}$$



$$D_n = A_n \oplus B_n \oplus C_n$$

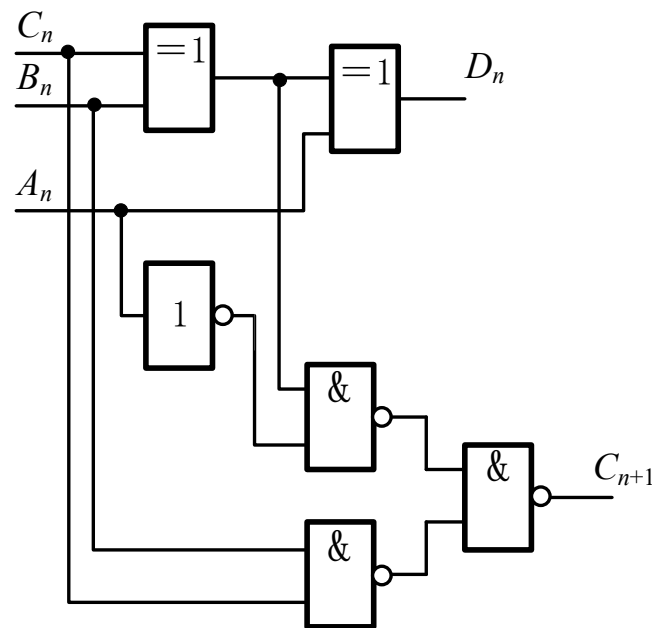
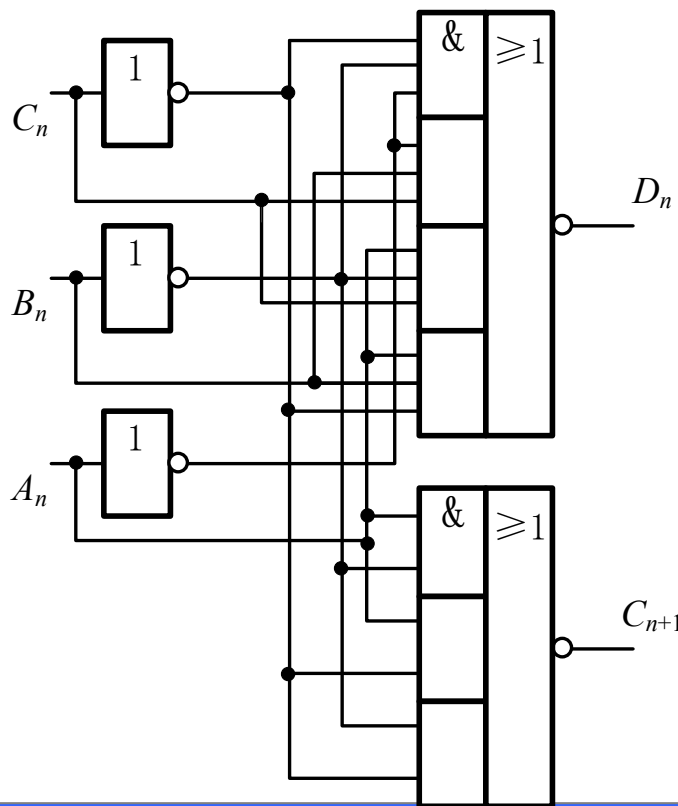
$$D_n = \overline{\overline{A_n} \overline{B_n} \overline{C_n} + \overline{A_n} B_n C_n + A_n B_n \overline{C_n} + A_n \overline{B_n} C_n}$$

$$C_{n+1} = \overline{\overline{B_n} \overline{C_n} + A_n \overline{C_n} + A_n \overline{B_n}}$$

$$C_{n+1} = \overline{\overline{A_n} \overline{B_n} C_n + \overline{A_n} B_n \overline{C_n} + B_n C_n}$$

$$= \overline{A_n} (B_n \oplus C_n) + B_n C_n$$

$$= \overline{\overline{\overline{A_n} (B_n \oplus C_n)} \cdot \overline{B_n C_n}}$$



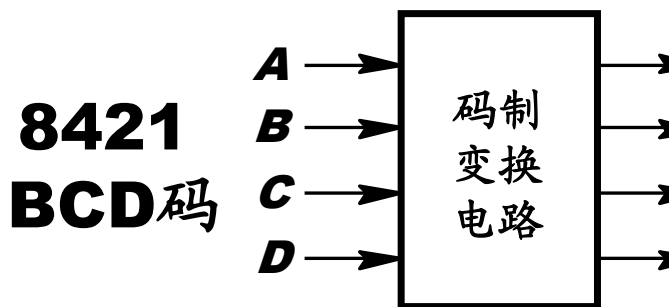


例2:用门电路设计一个将8421 BCD码转换为余3码的变换电路。

① 分析题意，逻辑抽象，列真值表。

该电路输入为8421 BCD码，输出为余3码，因此它是一个四输入、四输出的组合逻辑电路。

根据两种BCD码的编码规律，8421 BCD码不会出现1010~1111，因此这些输入组合为无关项。



A	B	C	D	E ₃	E ₂	E ₁	E ₀
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	×	×	×	×
1	0	1	1	×	×	×	×
1	1	0	0	×	×	×	×
1	1	0	1	×	×	×	×
1	1	1	0	×	×	×	×
1	1	1	1	×	×	×	×



② 选择器件，写出输出函数表达式。

题目没有具体指定用哪一种门电路，因此可以从门电路的数量、种类、速度等方面综合折衷考虑，选择最佳方案。

A	B	C	D	E_3	E_2	E_1	E_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	×	×	×	×
1	0	1	1	×	×	×	×
1	1	0	0	×	×	×	×
1	1	0	1	×	×	×	×
1	1	1	0	×	×	×	×
1	1	1	1	×	×	×	×

画K图、写表达式:

$AB \backslash CD$	00	01	11	10
00			X	1
01		1	X	1
11		1	X	X
10		1	X	X

E_3

$AB \backslash CD$	00	01	11	10
00		1	X	
01	1		X	1
11	1		X	X
10	1		X	X

E_2

$AB \backslash CD$	00	01	11	10
00	1	1	X	1
01			X	
11	1	1	X	X
10			X	X

E_1

$AB \backslash CD$	00	01	11	10
00	1	1	X	1
01			X	
11			X	X
10	1	1	X	X

E_0

$$E_3 = A + BC + BD = \overline{\overline{A} \cdot \overline{BC} \cdot \overline{BD}}$$

$$E_2 = B\overline{C}\overline{D} + \overline{B}C + \overline{B}D = B(\overline{C} + \overline{D}) + \overline{B}(C + D) = B \oplus (C + D)$$

$$E_1 = \overline{C}\overline{D} + CD = C \otimes D = C \oplus \overline{D}$$

$$E_0 = \overline{D}$$



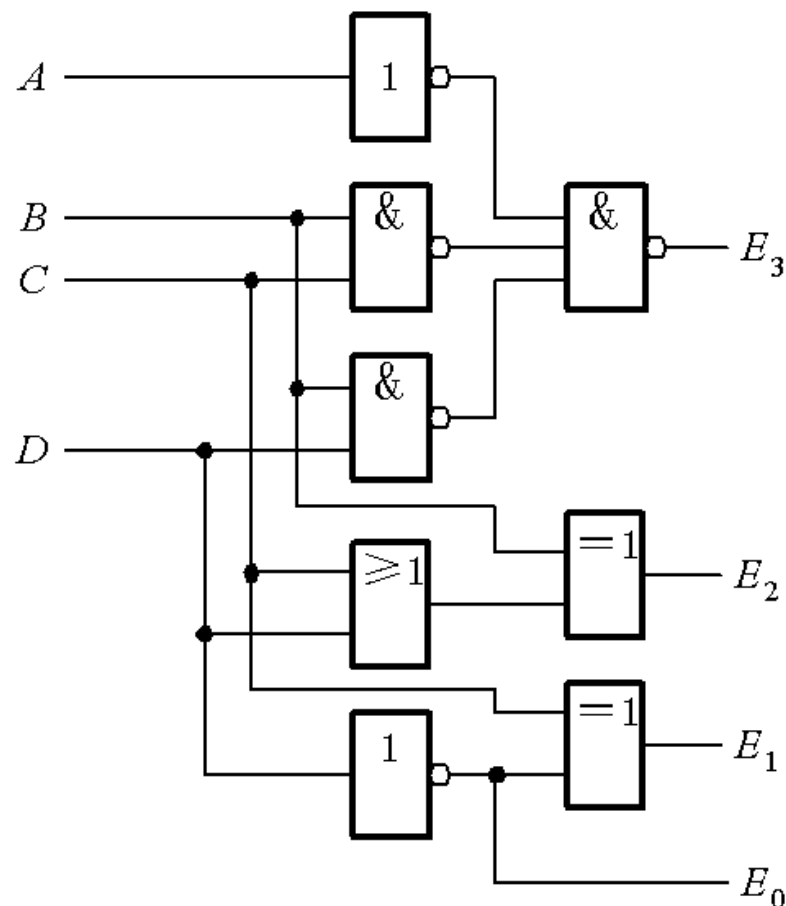
③ 画逻辑电路。

$$E_3 = A + BC + BD = \overline{\overline{A} \cdot \overline{BC} \cdot \overline{BD}}$$

$$E_2 = B\overline{C}\overline{D} + \overline{B}C + \overline{B}D = B(\overline{C} + \overline{D}) + \overline{B}(C + D) = B \oplus (C + D)$$

$$E_1 = \overline{C}\overline{D} + CD = C \otimes D = C \oplus \overline{D}$$

$$E_0 = \overline{D}$$



8421 BCD码转换为余3码的电路





组合电路设计的小结

- 1) 正确建立给定问题的**逻辑描述**是关键;
- 2) 工程考量, 指标兼顾: **电路简单**, 器件多见门类少, 级数少, 功耗小等;
- 3) 不同的逻辑表达式可能功能相同, 如

$$F(A, B, C, D) = \overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}BD$$

$$F(A, B, C, D) = \overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}CD$$



三、中规模集成（MSI）组合电路



集成逻辑门

- 逻辑门：完成一些基本逻辑功能的电子电路。现使用的主要为集成逻辑门。
- 集成逻辑门电路，是把门电路的所有元器件及连接导线制作在同一块半导体基片上构成的。



数字集成电路的分类

1. 按规模分类

小规模集成电路(**SSI**): 1~10门/片 或 1~100个元件/片;

中规模集成电路(**MSI**): 10~100门/片 或 100~1000个元件/片;

大规模集成电路(**LSI**): 100~10000门/片 或 1000~100000个元件/片;

超大规模集成电路(**VLSI**): 10000门/片以上或100000个元件/片以上.



常用MSI组合 逻辑器件:

☆ 译码器

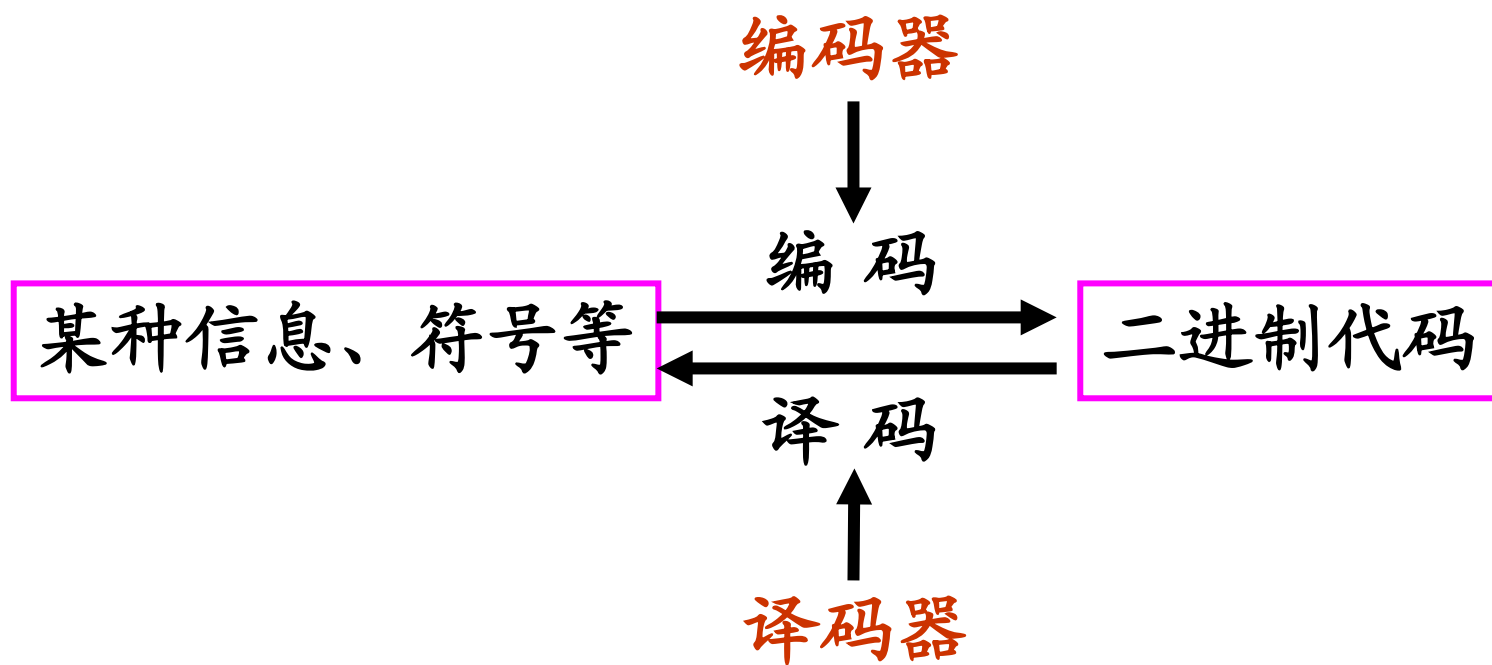
☆ 编码器

☆ 数据选择器
(MUX)

☆ 数据分配器

☆ 数码比较器

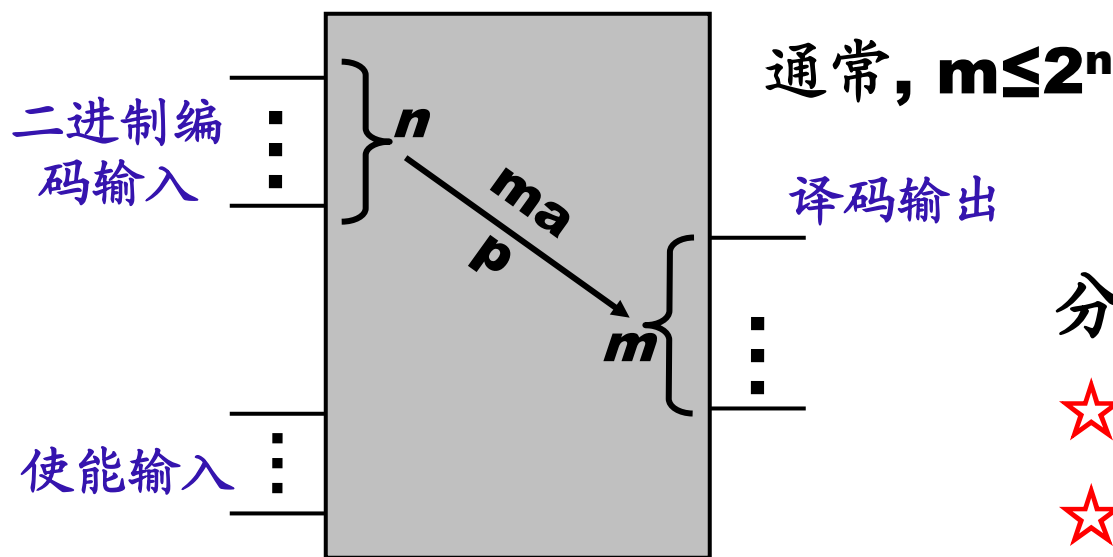
☆ 加法器减法器





一、译码器

译码是将某个二进制编码翻译成电路的某种状态，是将输入的某个二进制编码与电路输出的某种状态相对应。



MSI译码器的基本结构

分类:

- ☆ 二进制译码器
- ☆ 二—十进制译码器
- ☆ 显示译码器



(1) 二进制译码器

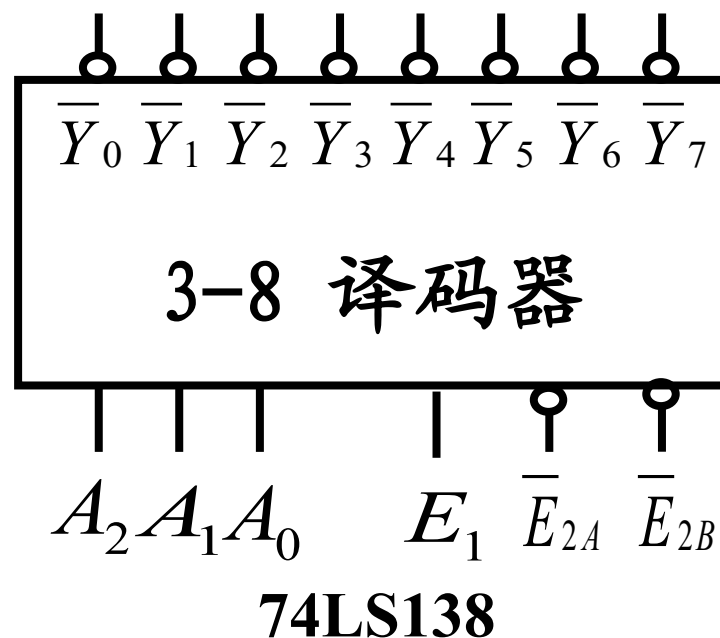
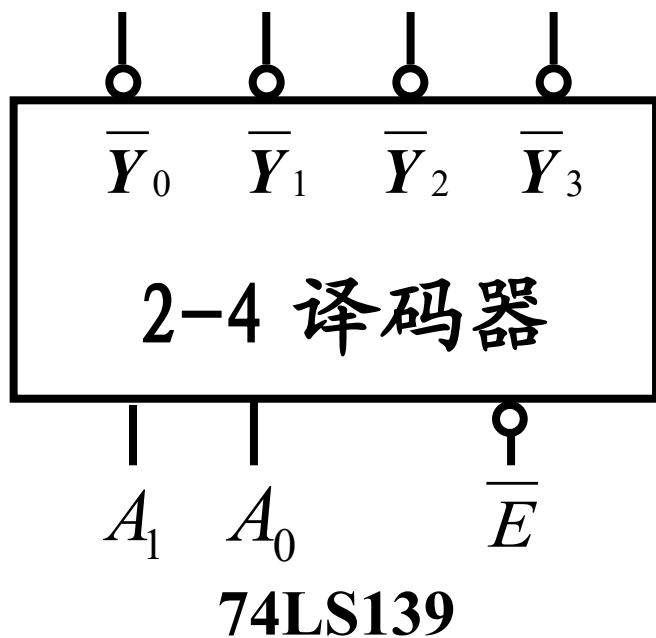
将 n 个输入的组合码译成 2^n 种电路状态。
也叫 n --- 2^n 译码器。

译码器的输入： 一组二进制代码

译码器的输出： 一组高低电平信号

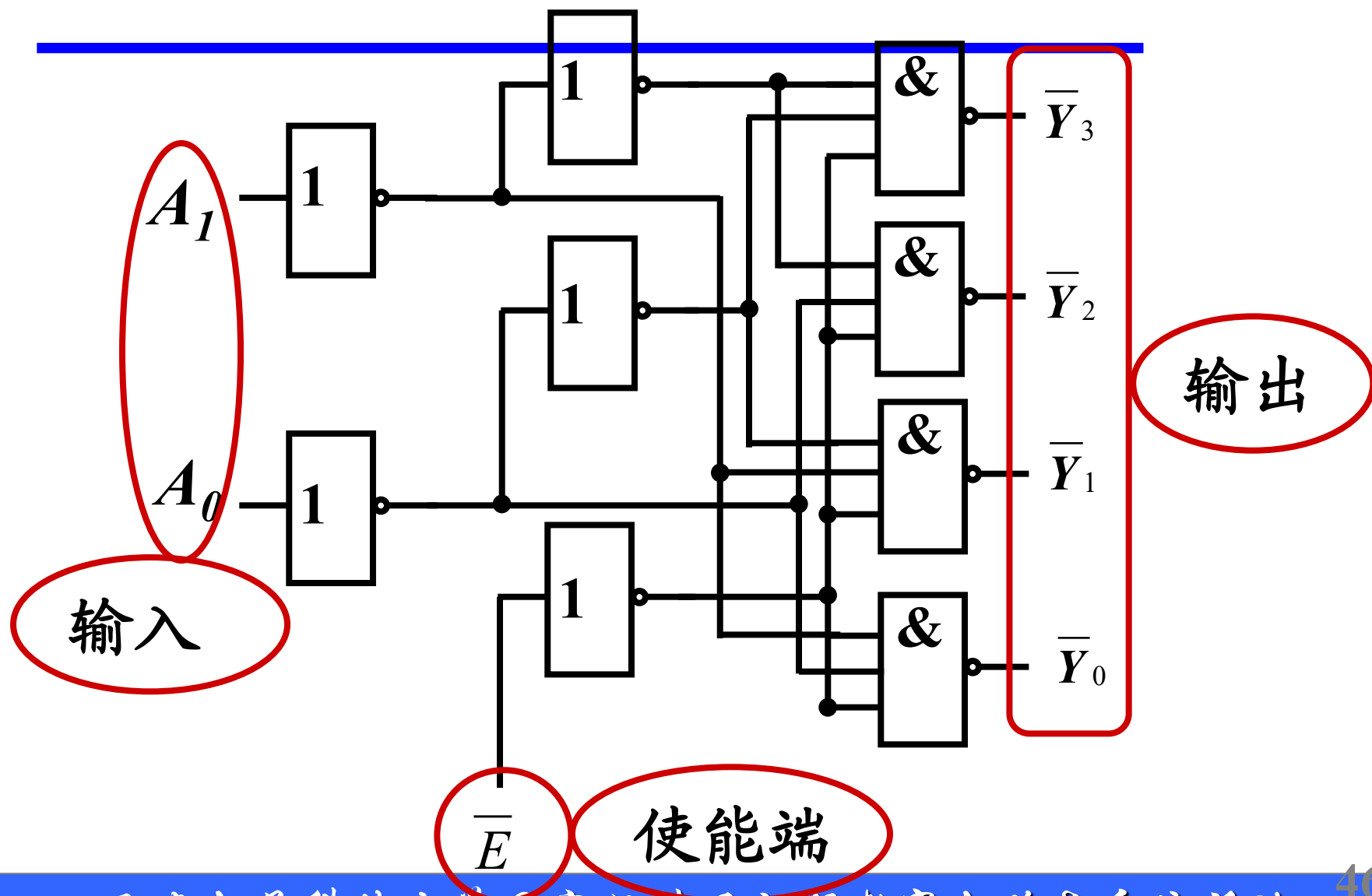


常用二进制译码器举例





*2-4 译码器74LS139的内部线路





*74LS139 2-4译码器的功能表

$\overline{E}$	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$
1	×	×	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

$$\overline{Y}_0 = \overline{\overline{A_1} \cdot \overline{A_0}}$$

$$\overline{Y}_2 = \overline{\overline{A_1} \cdot \overline{A_0}}$$

$$\overline{E} = 0 \text{ 时 } \overline{Y}_i = \overline{m_i}$$

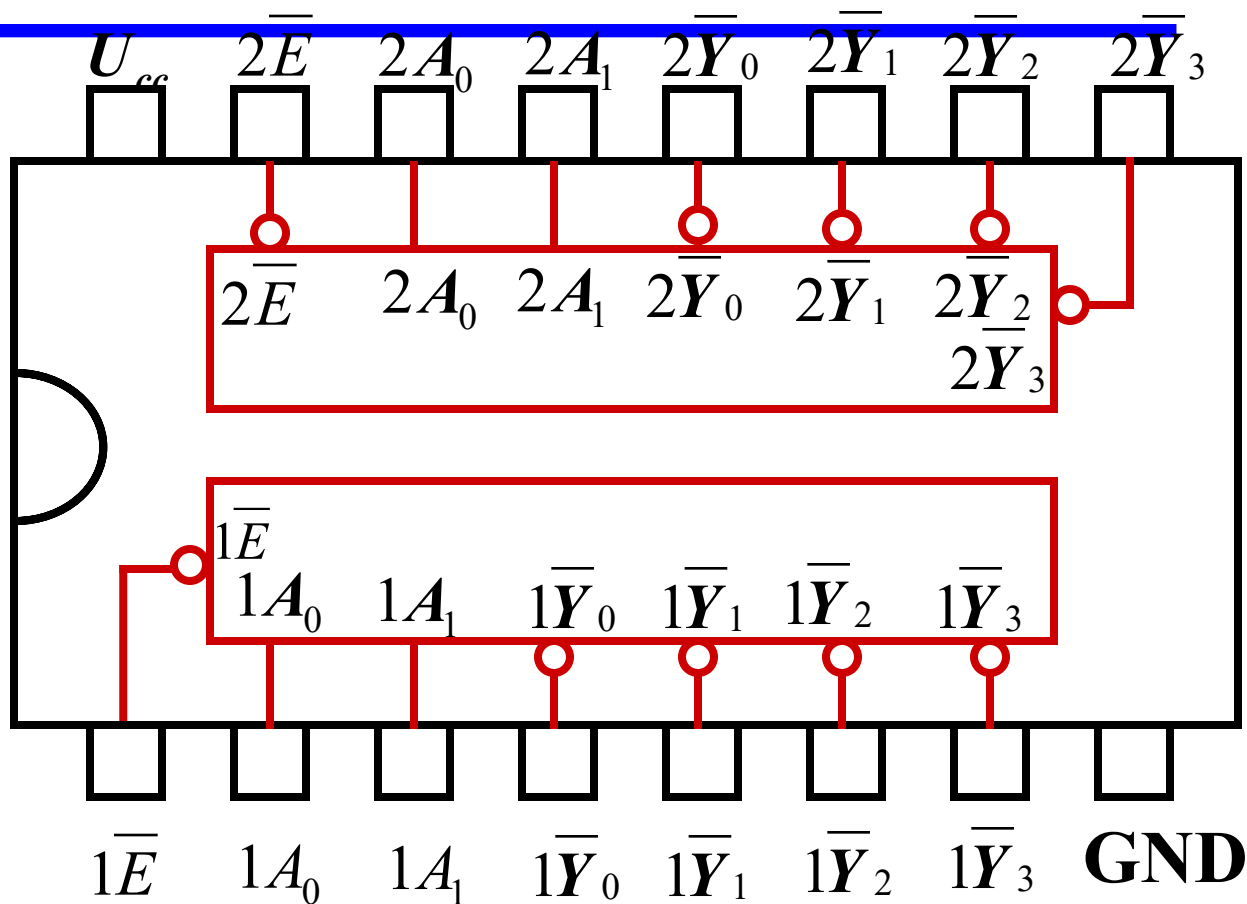
$$\overline{Y}_1 = \overline{\overline{A_1} \cdot A_0}$$

$$\overline{Y}_3 = \overline{A_1 \cdot A_0}$$

注意：译码器功能表要牢记



*74LS139管脚图



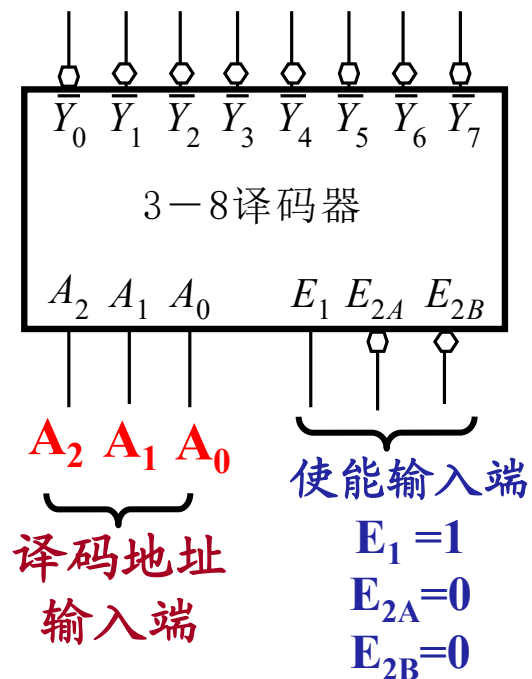
一片139种含两个2-4译码器

常用MSI译码器74138

3-8译码器功能表

3-8译码器的逻辑符号

译码输出，低电平有效



E_1	$E_{2A} + E_{2B}$	A_2	A_1	A_0	$\bar{Y}_0$	$\bar{Y}_1$	$\bar{Y}_2$	$\bar{Y}_3$	$\bar{Y}_4$	$\bar{Y}_5$	$\bar{Y}_6$	$\bar{Y}_7$
0	×	×	×	×	1	1	1	1	1	1	1	1
×	1	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

$$\bar{Y}_0 = \bar{A}_2 \bar{A}_1 \bar{A}_0$$

$$\bar{Y}_1 = \bar{A}_2 \bar{A}_1 A_0$$

$$\bar{Y}_2 = \bar{A}_2 A_1 \bar{A}_0$$

$$\bar{Y}_3 = \bar{A}_2 A_1 A_0$$

$$\bar{Y}_4 = \bar{A}_2 \bar{A}_1 \bar{A}_0$$

$$\bar{Y}_5 = \bar{A}_2 \bar{A}_1 A_0$$

$$\bar{Y}_6 = \bar{A}_2 A_1 \bar{A}_0$$

$$\bar{Y}_7 = A_2 A_1 A_0$$

二进制译码器的输出将输入二进制代码的各种组合状态都译出来了。因此，二进制译码器又称全译码器，它的输出提供了输入变量的全部最小项。



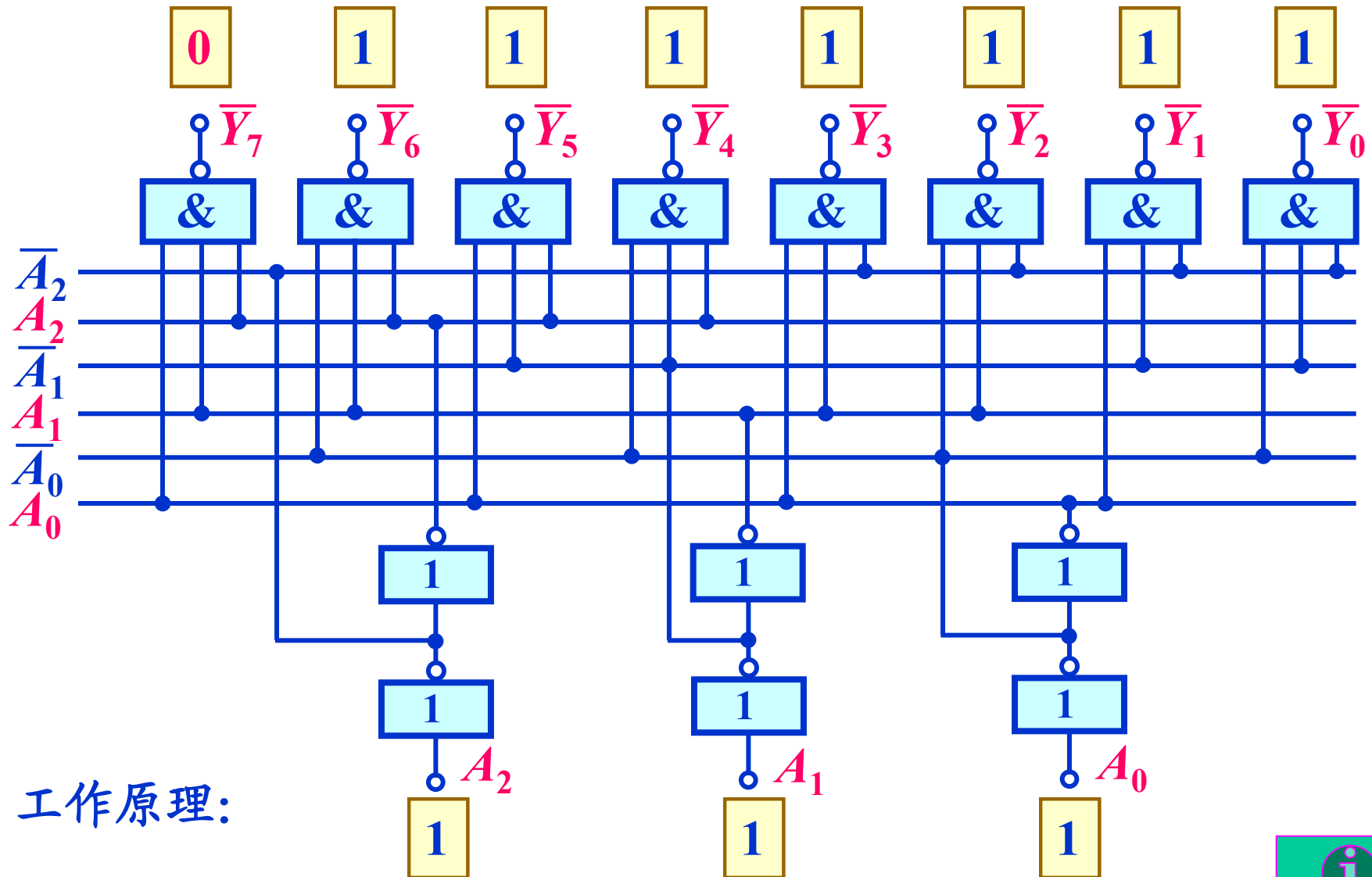
二进制译码器

二进制译码器的输出方程可表示为：

$$\overline{Y}_i = \overline{m}_i = M_i \quad (i = 0 \dots 2^n - 1)$$

可见，当译码器使能端有效时，输出函数正好与译码地址输入变量的最小项（或最大项）联系起来。

3-to-8 译码器内部等效电路——低电平输出有效



二进制译码器的应用很广，典型的应用有以下几种：

- ① 实现逻辑函数；
- ② 实现设备及存储系统的地址译码；
- ③ 带使能端的译码器可用作数据分配器或脉冲分配器。

译码器应用： ① 实现逻辑函数

【例 4.3.1】 试用3-8译码器实现函数：

$$F_1 = \sum m(0, 4, 7)$$

$$F_2 = \sum m(1, 2, 3, 5, 6, 7)$$

二进制译码器的输出方程可表示为：

$$\overline{Y}_i = \overline{m}_i = M_i \quad (i = 0 \cdots 2^{n-1})$$

$$F_1(A, B, C) = \sum m(0, 4, 7)$$

$$F_2(A, B, C) = \sum m(1, 2, 3, 5, 6, 7)$$

解: $\bar{Y}_i = \bar{m}_i = M_i \quad (i = 0 \dots 2^n - 1)$

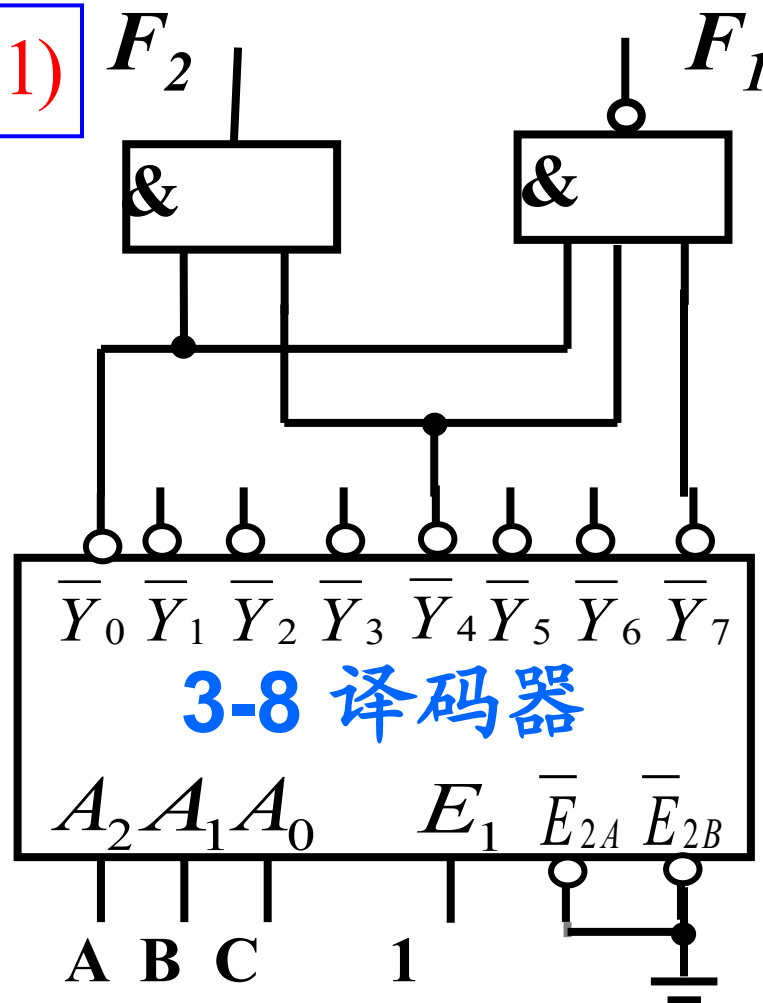
$$F_1(A, B, C) = \sum m(0, 4, 7)$$

$$= m_0 + m_4 + m_7$$

$$= \overline{\overline{m_0} \cdot \overline{m_4} \cdot \overline{m_7}}$$

$$\overline{F_2} = m_0 + m_4$$

$$F_2 = \overline{\overline{F_2}} = \overline{m_0 + m_4} = \overline{m_0} \cdot \overline{m_4} = \bar{Y}_0 \cdot \bar{Y}_4$$



译码器应用： ① 实现逻辑函数

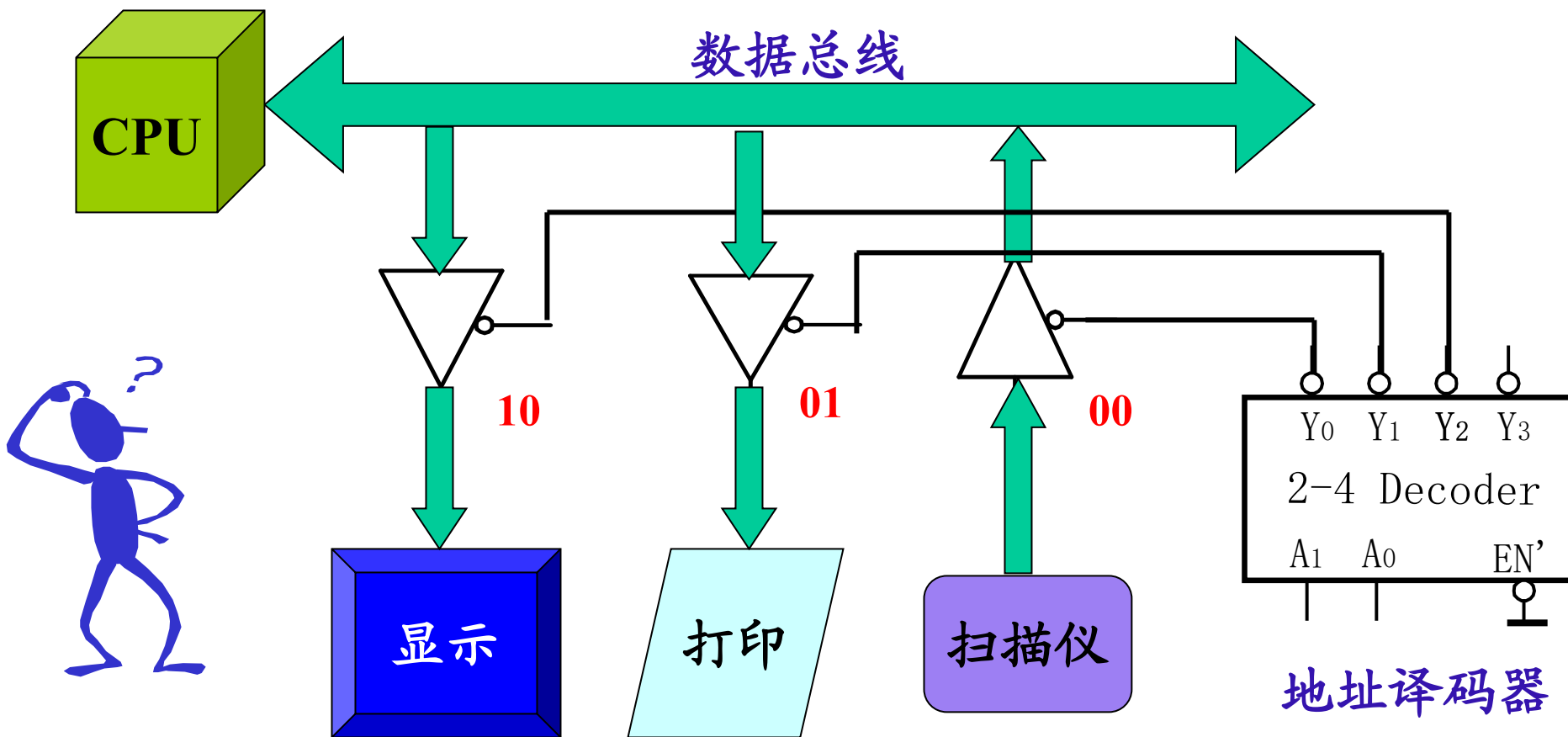
- **Ex.:** 用74138和门电路实现下面的逻辑函数。

$$f_1(a,b,c) = \sum m(1,2,4,5)$$

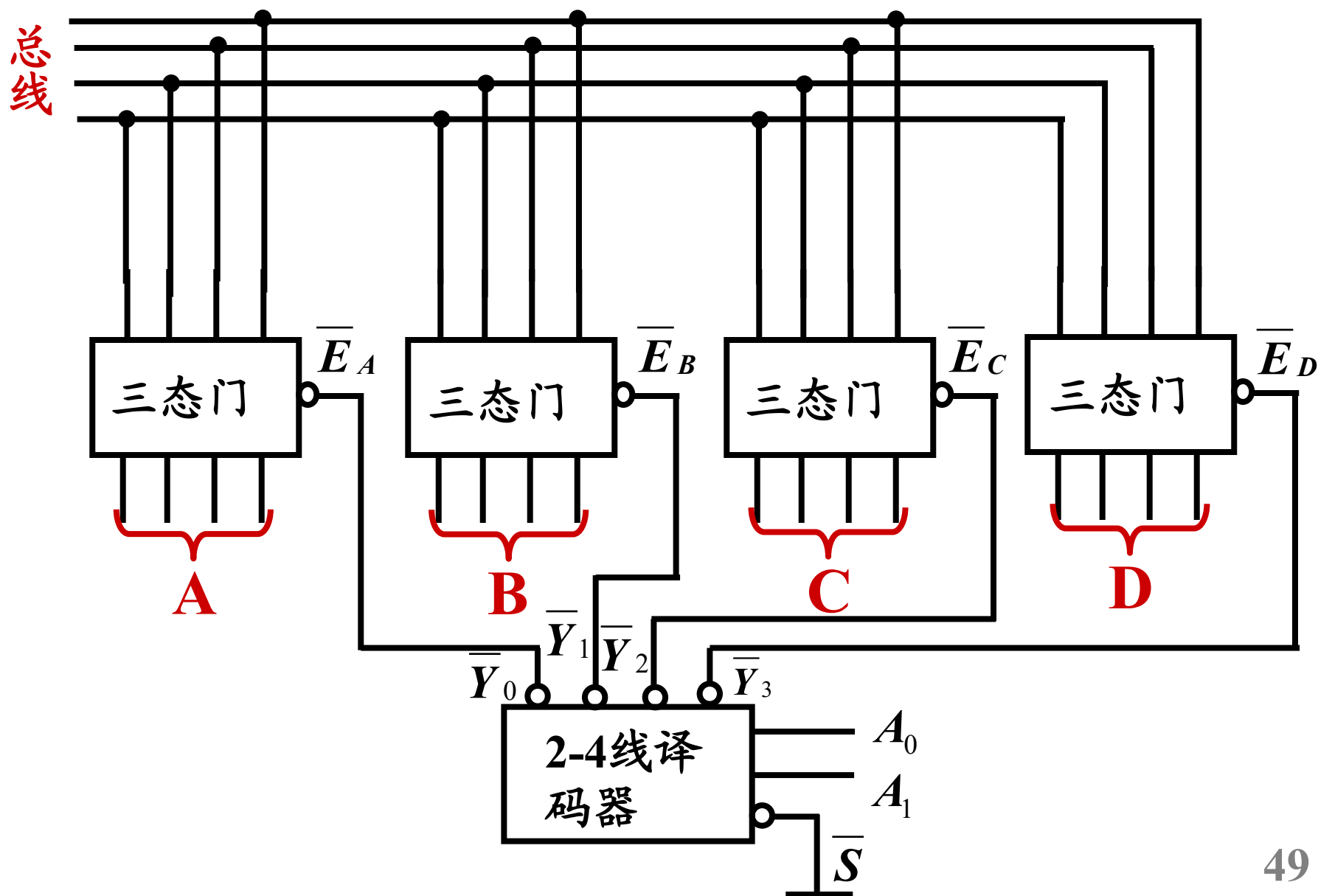
$$f_2(a,b,c) = \prod M(2,3,6,7)$$

- **Ex.:** 用74138和门电路设计一个一位二进制全加器。

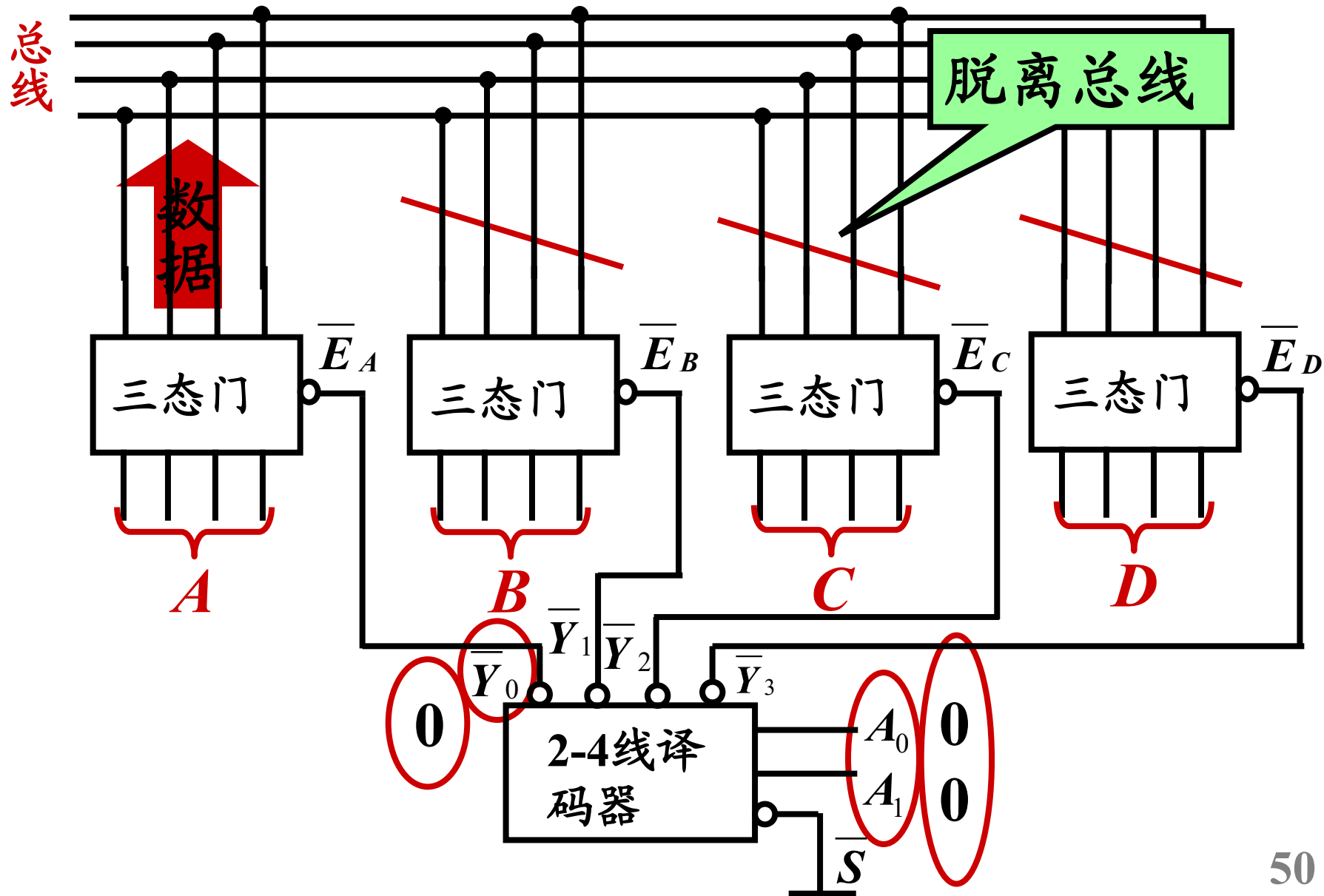
译码器应用：② 实现设备的地址译码



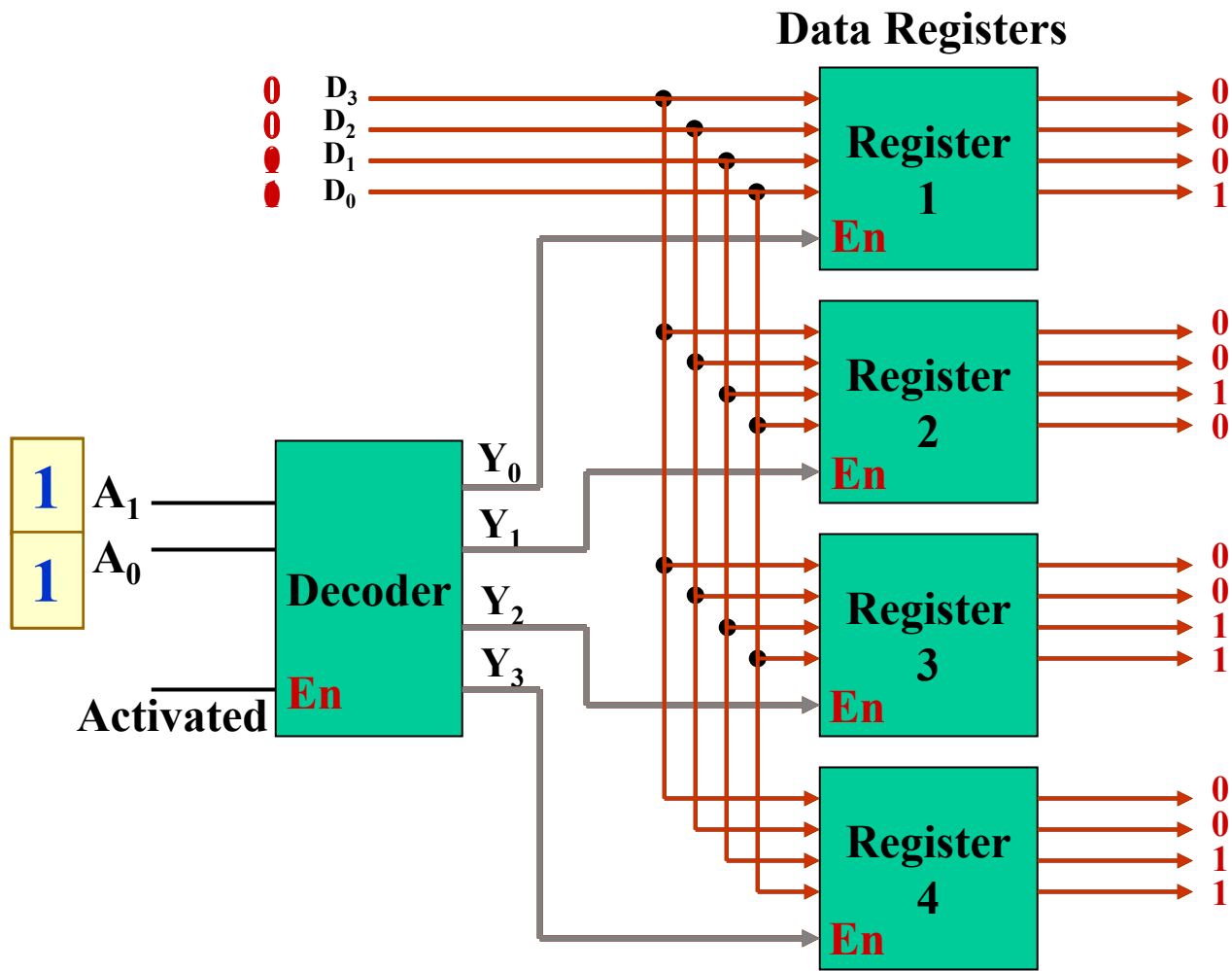
*例：利用译码器分时将采样数据送入计算机。



*工作原理: (以 $A_0A_1=00$ 为例)

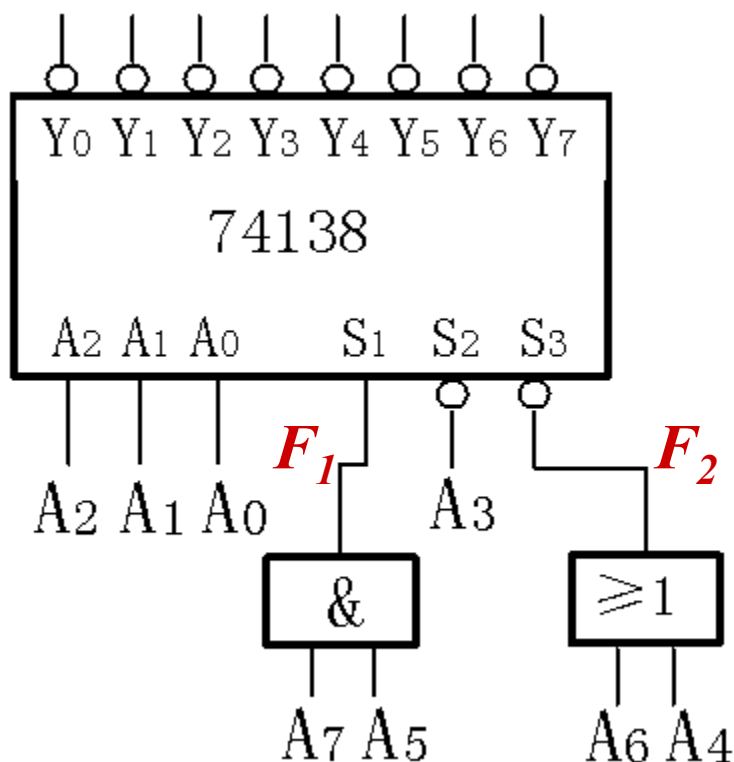


译码器应用： ② 实现设备控制的地址译码



译码器应用举例

- 例1: 分析下图所示地址译码电路中, $Y_0 \sim Y_7$ 的有效译码地址是_____。



74138正常译码时:

$F_1=1$, and $A_3=0$, and $F_2=0$.

$$F_1=A_5A_7=1 \Rightarrow A_5=A_7=1$$

$$F_2=A_4+A_6=0 \Rightarrow A_4=A_6=0$$

A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
1	0	1	0	0	x	x	x

故, 该电路的有效地址范围是:

$(10100000)_2 \sim (10100111)_2$

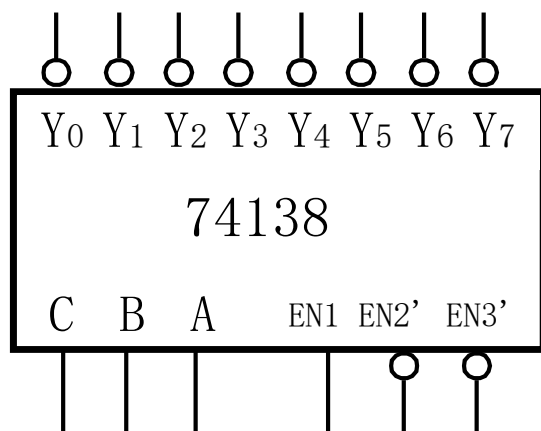
or $(A0H \sim A7H)$

- **例2:** 用3-8译码器设计一个地址译码电路，要求有效的地址译码范围为2E0H~2E7H。

分析: 2E0H~2E7H, 即: 10 1110 0000B ~ 10 1110 0111B

假设地址: $A_9A_8A_7A_6A_5A_4A_3A_2A_1A_0$

有: $A_9=A_7=A_6=A_5=1$
 $A_8=A_4=A_3=0$



A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
1	0	1	1	1	0	0	0	0	0
1	0	1	1	1	0	0	0	0	1
1	0	1	1	1	0	0	0	1	0
1	0	1	1	1	0	0	0	1	1
1	0	1	1	1	0	0	1	0	0
1	0	1	1	1	0	0	1	0	1
1	0	1	1	1	0	0	1	1	0
1	0	1	1	1	0	0	1	1	1

如何实现该地址译码电路?



扩展:

二进制译码器的扩展应用

补充：二进制译码器的扩展应用

利用译码器的**使能端**可以实现多片译码器功能的扩展。

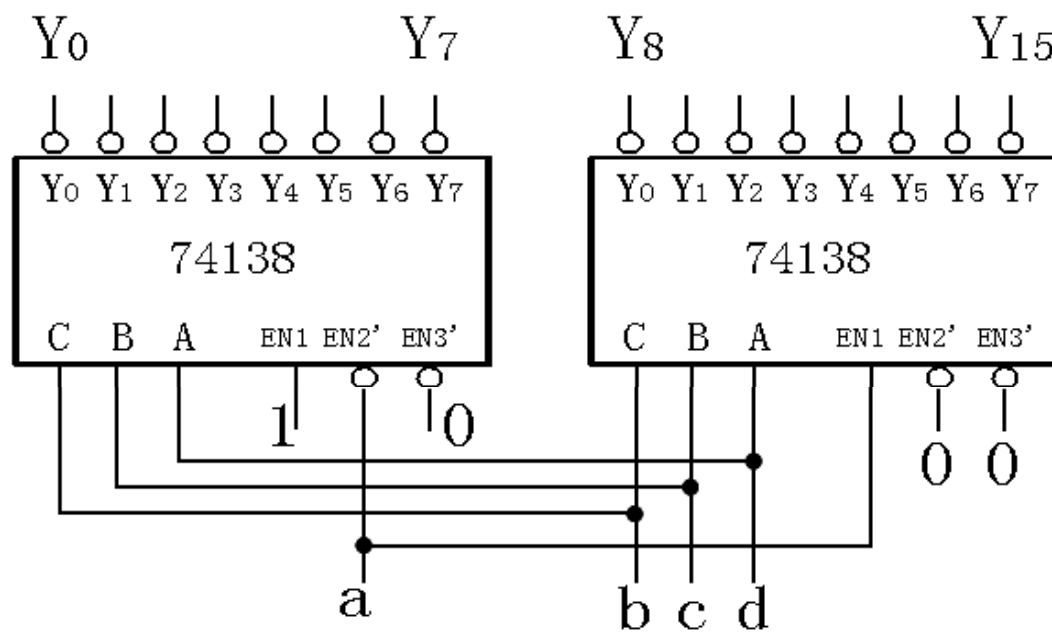
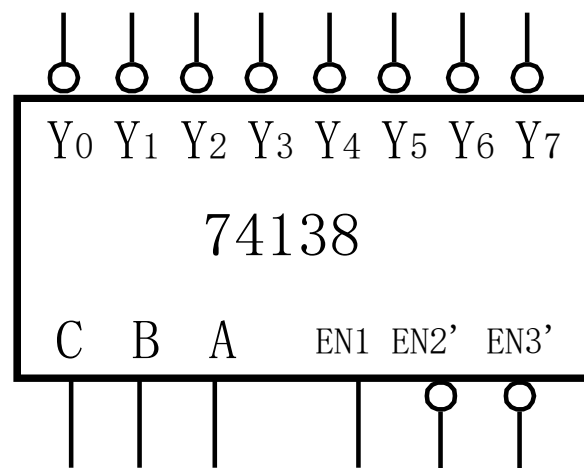
例：使用3-8译码器来实现4-16译码功能。 (**输出要求为低有效**)

我们有

3—8 译码器：	3-input	8-output
4—16 译码器：	4-input	16-output

分析4-16译码器的等效功能表

a	b	c	d	Y
0	0	0	0	Y_0
0	0	0	1	Y_1
0	0	1	0	Y_2
0	0	1	1	Y_3
0	1	0	0	Y_4
0	1	0	1	Y_5
0	1	1	0	Y_6
0	1	1	1	Y_7
1	0	0	0	Y_8
1	0	0	1	Y_9
1	0	1	0	Y_{10}
1	0	1	1	Y_{11}
1	1	0	0	Y_{12}
1	1	0	1	Y_{13}
1	1	1	0	Y_{14}
1	1	1	1	Y_{15}



思考：

1、如何用2-4译码器实现4-16译码器功能？

（多片扩展）

2、用3-8译码器实现下面的函数：

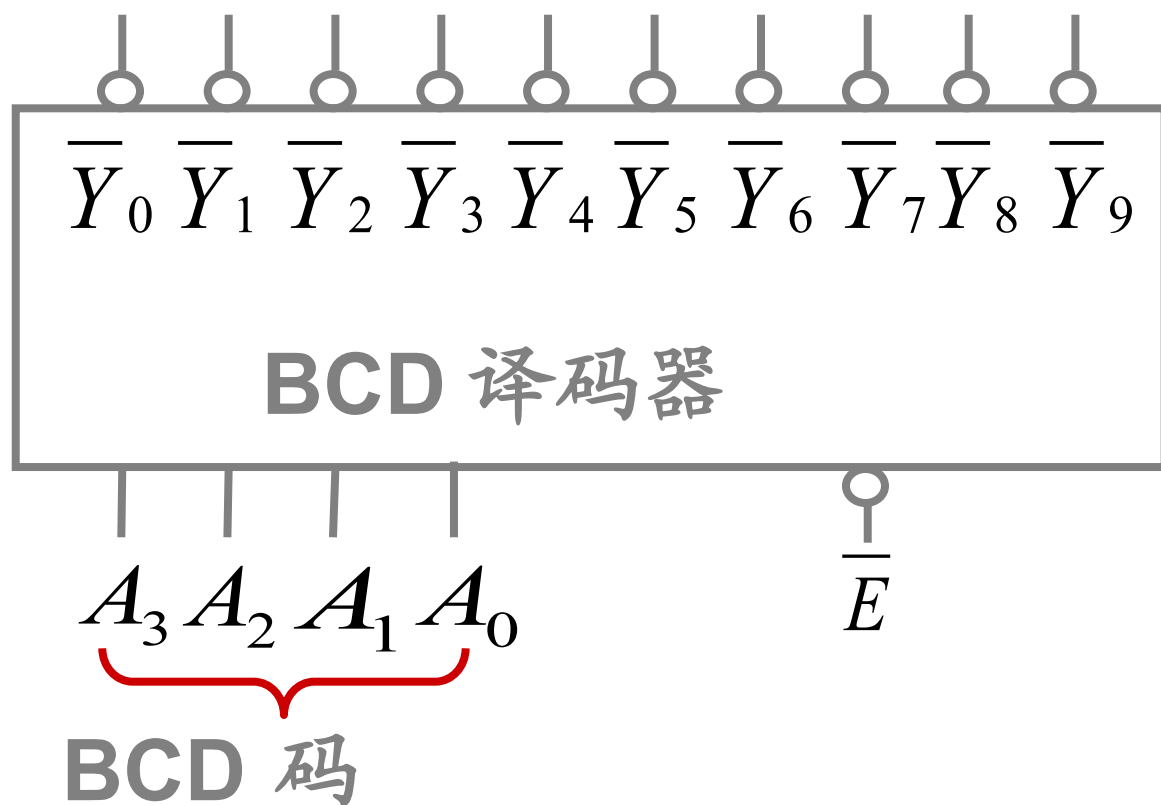
$$F = \sum m(8,9,12,14,15)$$

（输入变量数超过地址输入位数）

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>Y</i>
0	0	0	0	Y_0
0	0	0	1	Y_1
0	0	1	0	Y_2
0	0	1	1	Y_3
0	1	0	0	Y_4
0	1	0	1	Y_5
0	1	1	0	Y_6
0	1	1	1	Y_7
1	0	0	0	Y_8
1	0	0	1	Y_9
1	0	1	0	Y_{10}
1	0	1	1	Y_{11}
1	1	0	0	Y_{12}
1	1	0	1	Y_{13}
1	1	1	0	Y_{14}
1	1	1	1	Y_{15}

(2) 二-十进制译码器 (BCD译码器)

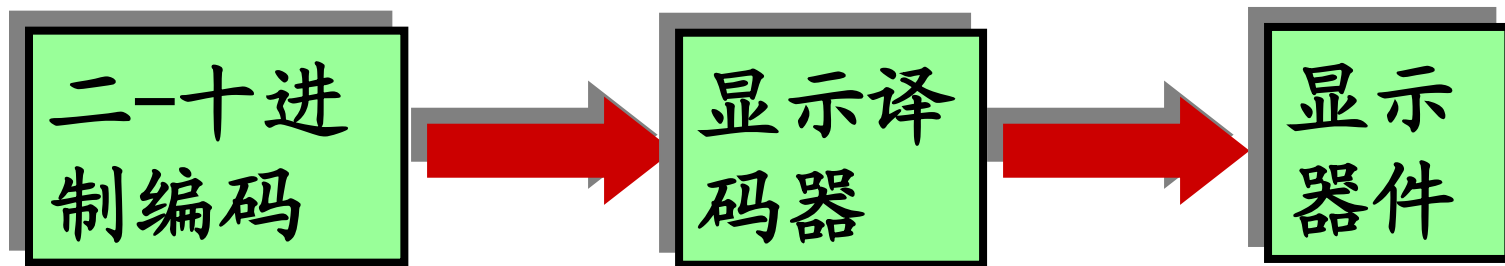
将输入的一位BCD码（四位二进制数）译成10种不同的电路状态。





(3) 显示译码器

在数字系统中，常常需要将运算结果用人们习惯的十进制显示出来，这就要用到**显示译码器**。





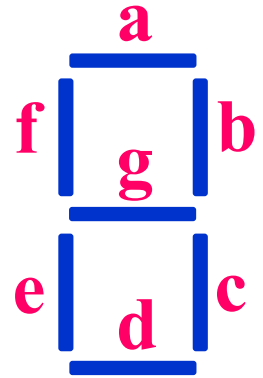
显示译码器

显示译码器是用来驱动显示器件，以显示数字或字符的MSI部件。

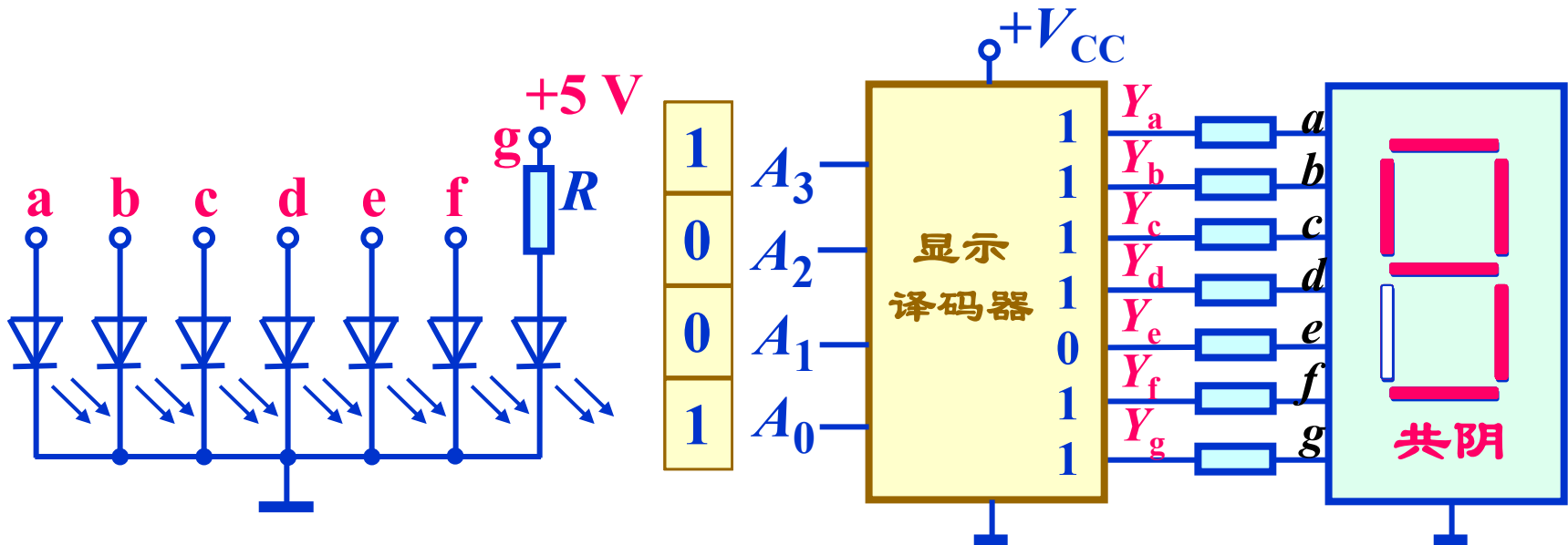
显示译码器随显示器件的类型而异，与辉光数码管相配的是BCD十进制译码器，而常用的**发光二极管(LED)数码管**、**液晶数码管**、**荧光数码管**等是由7个或8个字段构成字形的，因而与之相配的有BCD七段或BCD八段显示译码器。

Display Decoder

——Seven-segment display



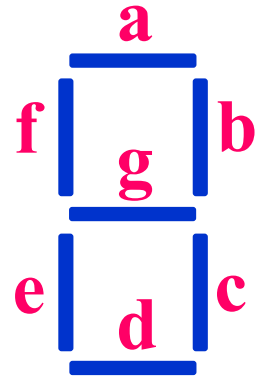
Common Cathode (共阴极)



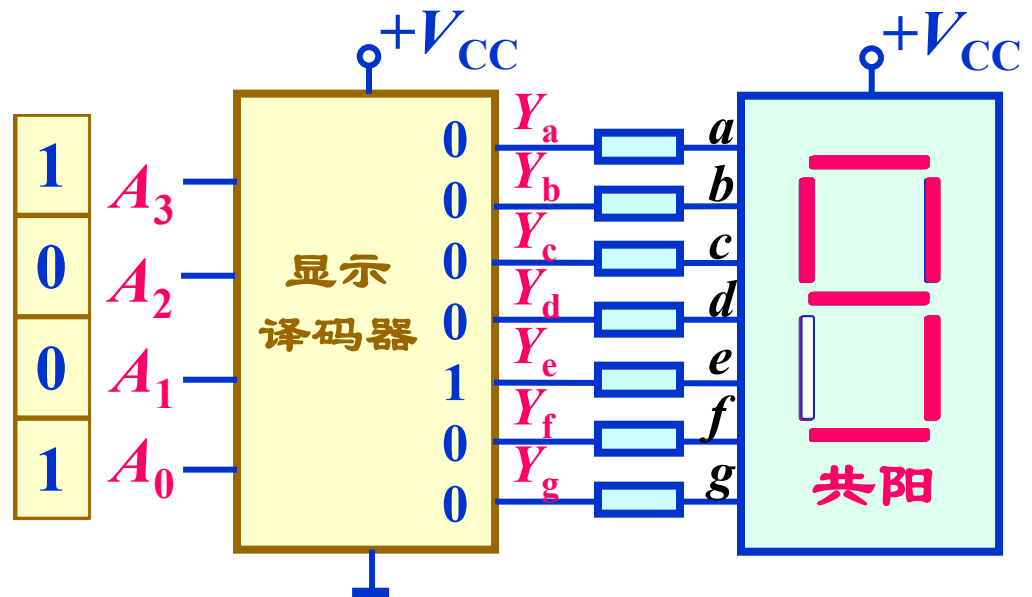
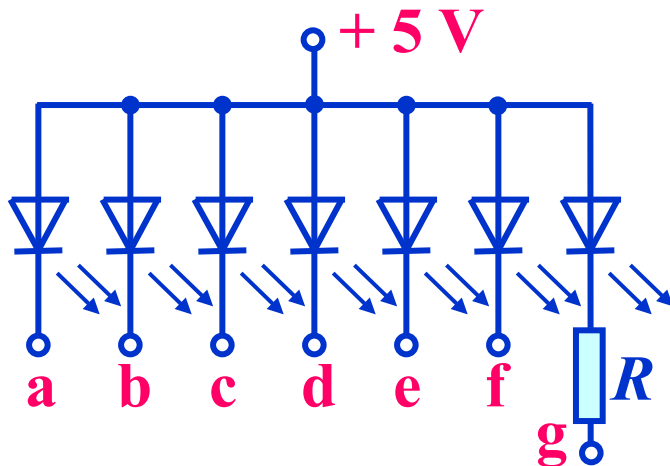
principle of work:

Display Decoder

——Seven-segment display



Common Anode (共阳极)



principle of work:

表 4-12 共阴极8421BCD七段译码器真值表

输 入				输 出							字 形
D	C	B	A	F_a	F_b	F_c	F_d	F_e	F_f	F_g	
0	0	0	0	1	1	1	1	1	1	0	. □
0	0	0	1	0	1	1	0	0	0	0	—
0	0	1	0	1	1	0	1	1	0	1	□
0	0	1	1	1	1	1	1	0	0	1	□
0	1	0	0	0	1	1	0	0	1	1	□
0	1	0	1	1	0	1	1	0	1	1	□
0	1	1	0	1	0	1	1	1	1	1	□
0	1	1	1	1	1	1	0	0	0	0	□
1	0	0	0	1	1	1	1	1	1	1	□
1	0	0	1	1	1	1	1	0	1	1	□



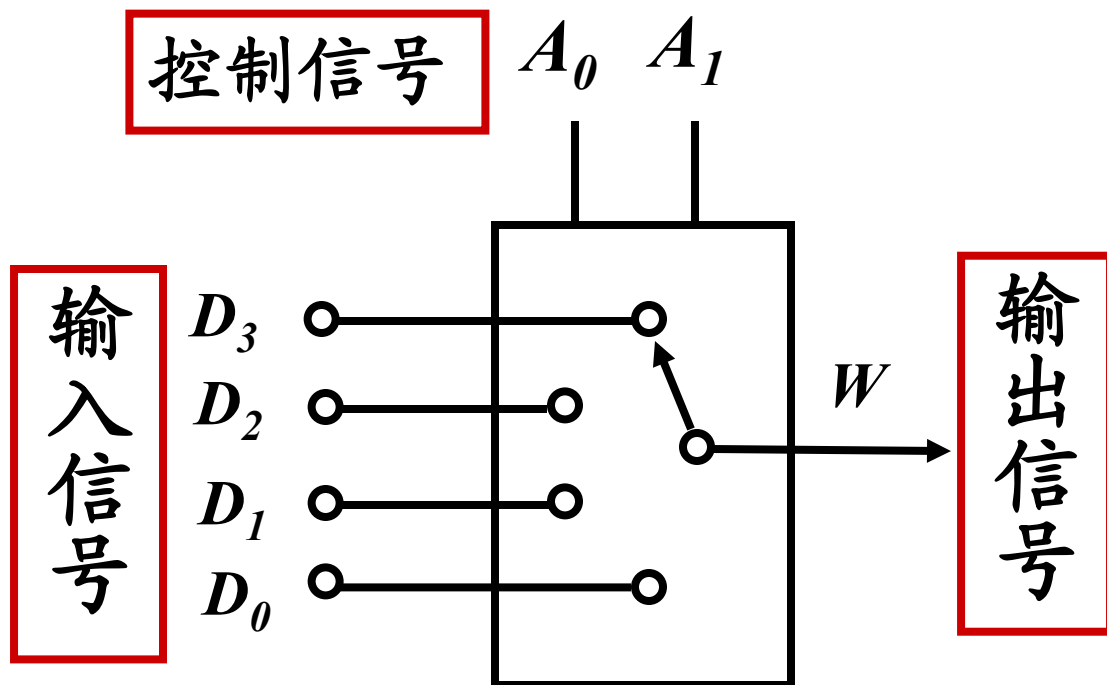
译码器小结

- 译码器功能及输出方程
- 译码器设计电路
 1. 实现逻辑函数;
 2. 实现设备及存储系统的地址译码;
 3. 译码器扩展;
- 显示译码器



二、数据选择器 (Multiplexer, 简称MUX)

从一组数据中选择一路信号进行传输的电路，称为**数据选择器**。

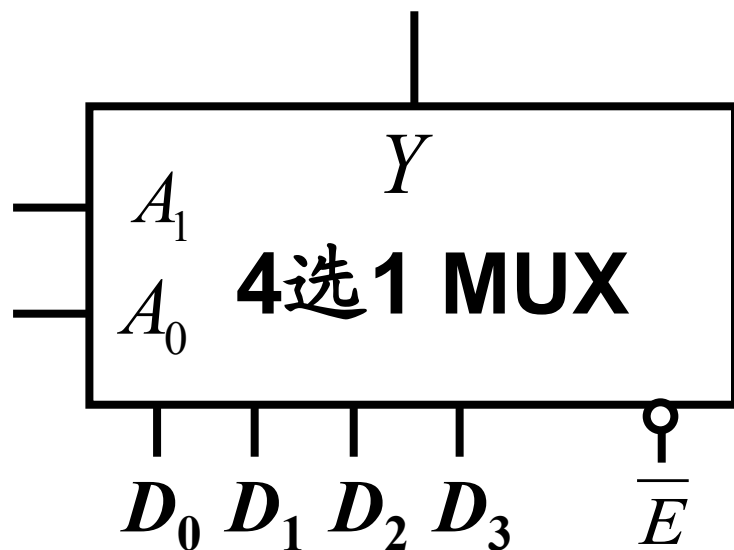


原理示意图

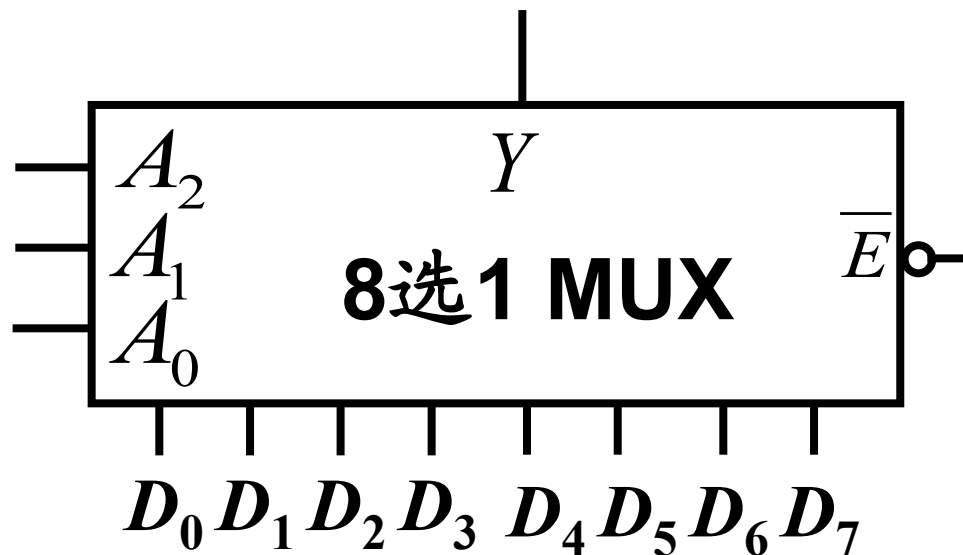
数据选择器类似一个多掷开关。选择哪一路信号由相应的一组控制信号控制。



常用数据选择器举例



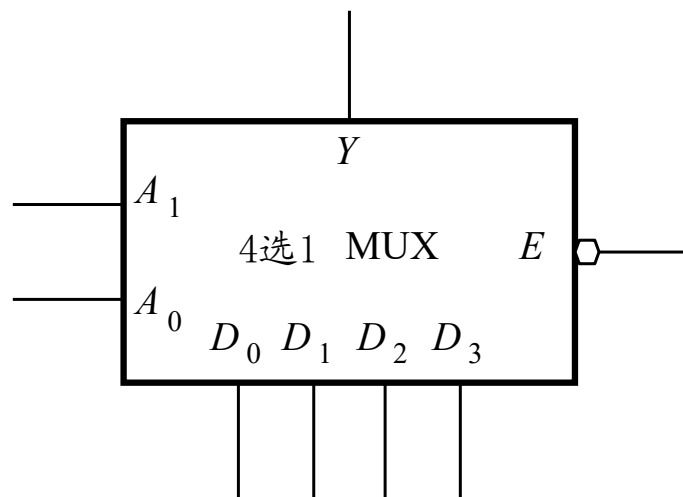
74LS153



74LS151



4选1 MUX的逻辑符号、功能表及输出表达式。



4选1 MUX的逻辑符号

4选1 MUX功能表

E	A_1	A_0	Y
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	×	×	0

输出Y的表达式？

当 $E=0$ 时，4选1 MUX的逻辑功能还可以用表达式表示：

$$Y = \bar{A}_1 \bar{A}_0 D_0 + \bar{A}_1 A_0 D_1 + A_1 \bar{A}_0 D_2 + A_1 A_0 D_3 = \sum_{i=0}^3 m_i D_i$$



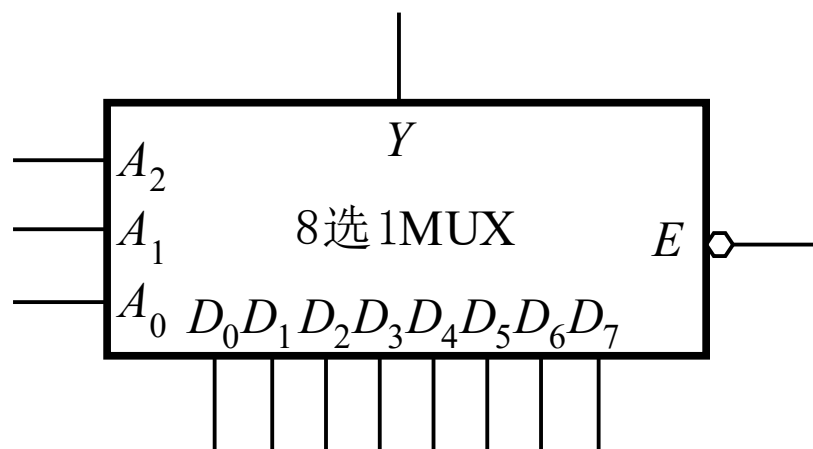
$$Y = \overline{A_1}\overline{A_0}D_0 + \overline{A_1}A_0D_1 + A_1\overline{A_0}D_2 + A_1A_0D_3 = \sum_{i=0}^3 m_i D_i$$

式中， m_i 是地址变量 A_1 、 A_0 所对应的最小项，称地址最小项。用矩阵形式表示为

$$Y = (\overline{A_1}\overline{A_0} \quad \overline{A_1}A_0 \quad A_1\overline{A_0} \quad A_1A_0) \begin{bmatrix} D_0 \\ D_1 \\ D_2 \\ D_3 \end{bmatrix} = (A_1A_0)_m (D_0D_1D_2D_3)^T$$



8选1 MUX的逻辑符号、功能表及输出表达式。



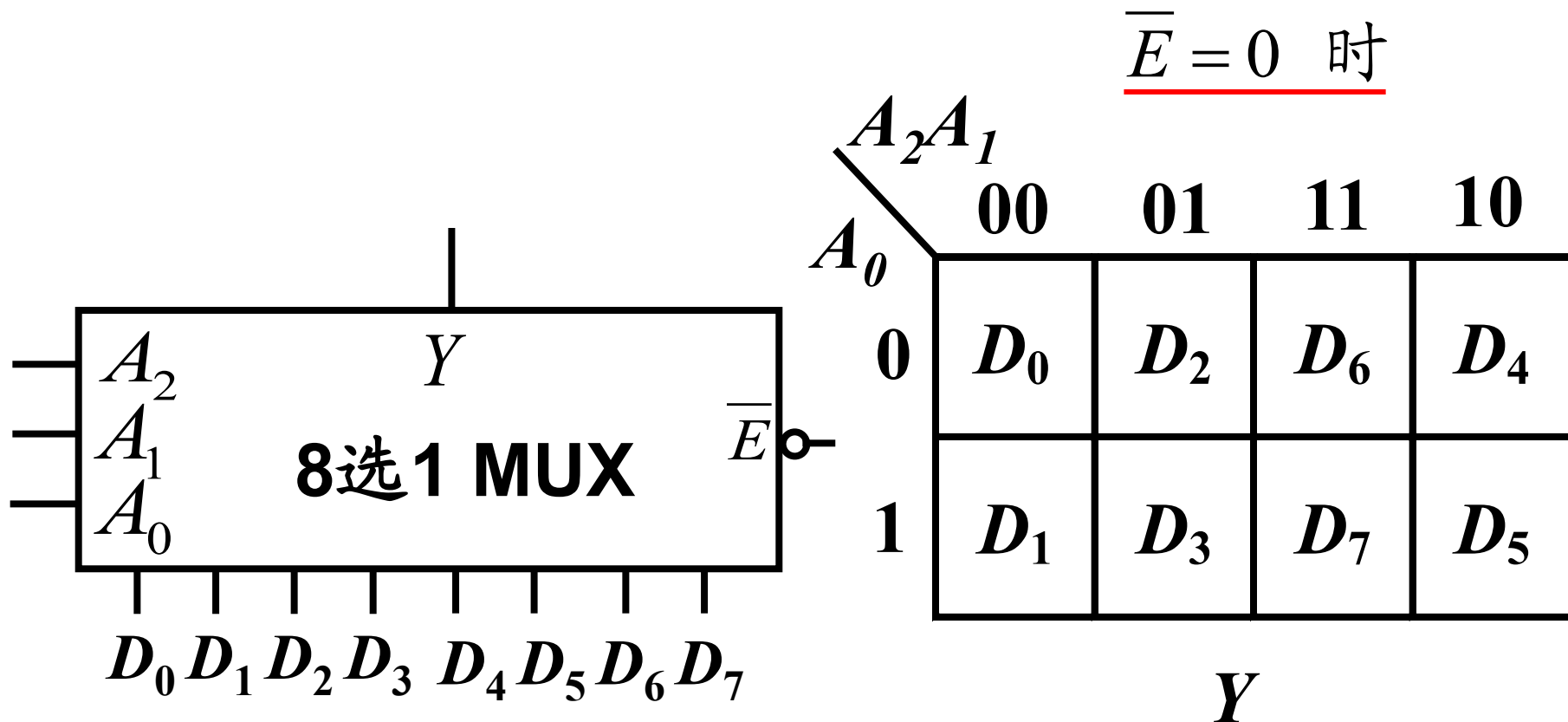
E	A_2	A_1	A_0	Y
1	×	×	×	0
0	0	0	0	D_0
0	0	0	1	D_1
0	0	1	0	D_2
0	0	1	1	D_3
0	1	0	0	D_4
0	1	0	1	D_5
0	1	1	0	D_6
0	1	1	1	D_7

$$Y = \sum_{i=0}^7 m_i D_i$$

$$= (A_2 A_1 A_0)_m (D_0 D_1 D_2 D_3 D_4 D_5 D_6 D_7)^T$$



8选1 MUX的性质 (K图)

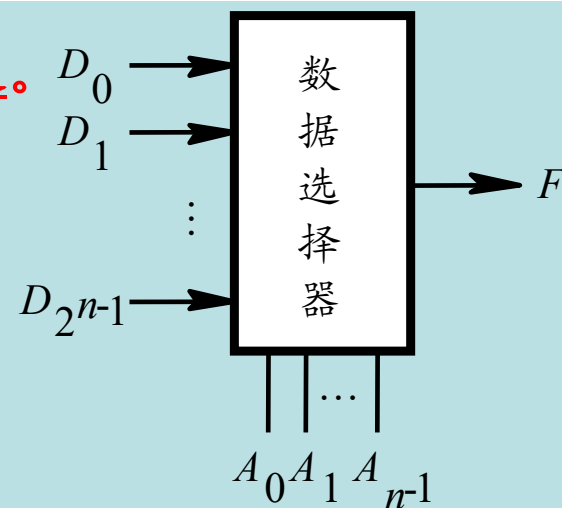




数据选择器的应用

数据选择器的应用很广，典型应用有：

- ① 作数据选择，以实现多路信号分时传送。
- ② 实现组合逻辑函数。
- ③ 在数据传输时实现并—串转换。
- ④ 产生序列信号。





例: 用 8-选-1 MUX 实现

$$F = \sum m(1, 2, 3, 4, 5, 7)$$

从 F 可以看出, 该逻辑函数
一个 8-to-1 MUX 有 3 个地址

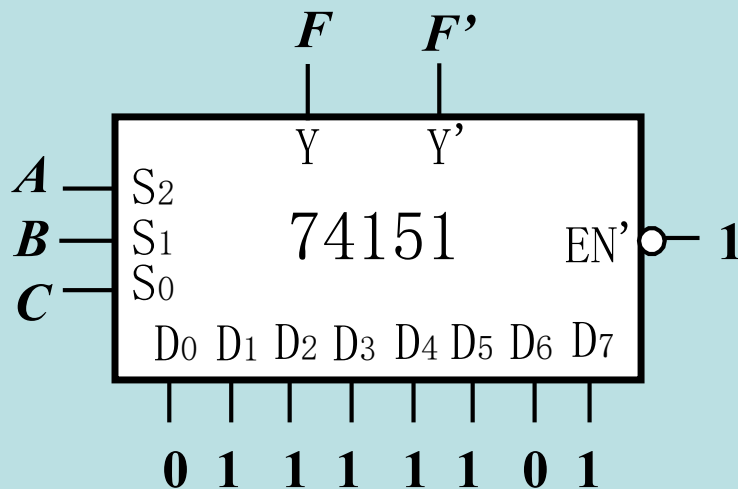
1) 函数对比法 (代数法)

$$Y = \sum_{i=0}^7 m_i D_i = (A_2 A_1 A_0)(D_0 D_1 D_2 \dots D_7)$$

$$= m_0 \cdot D_0 + m_1 \cdot D_1 + m_2 \cdot D_2 + m_3 \cdot D_3 + m_4 \cdot D_4 + m_5 \cdot D_5 + m_6 \cdot D_6 + m_7 \cdot D_7$$

$$F = \sum m(1, 2, 3, 4, 5, 7)$$

$$= m_0 \cdot 0 + m_1 \cdot 1 + m_2 \cdot 1 + m_3 \cdot 1 + m_4 \cdot 1 + m_5 \cdot 1 + m_6 \cdot 0 + m_7 \cdot 1$$





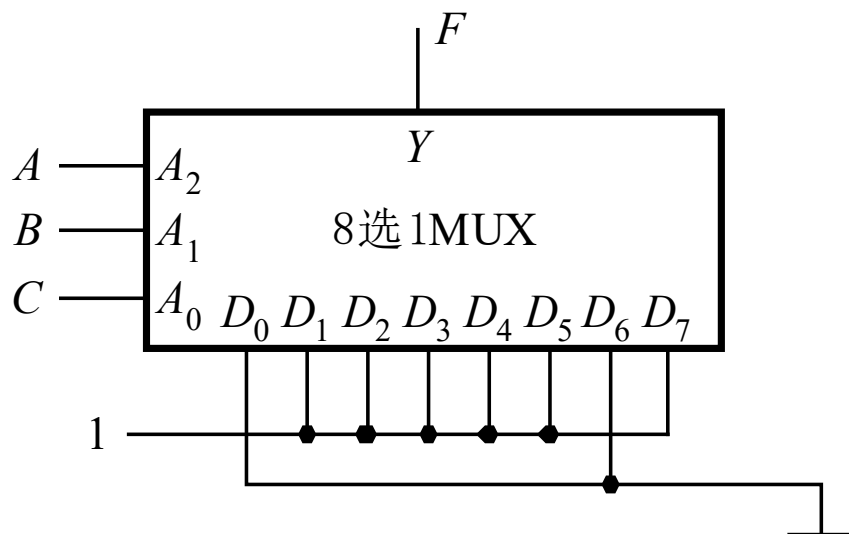
例：试用8选1MUX实现逻辑函数：

$$F = \overline{A}B + A\overline{B} + C$$

2) 卡诺图对比法

AB		C			
		00	01	11	10
C	0	0	1	0	1
	1	1	1	1	1

A_2A_1		A_0			
		00	01	11	10
A_0	0	D_0	D_2	D_6	D_4
	1	D_1	D_3	D_7	D_5



需要注意的是，因为函数F中各最小项的标号是按A、B、C的权为4、2、1写出的，因此A、B、C必须依次加到 A_2 、 A_1 、 A_0 端。



MUX应用：实现逻辑函数

对于 n 个地址输入的MUX，其表达式为

$$Y = \sum_{i=0}^{2^n-1} m_i D_i$$

其中 m_i 是由地址变量 A_{n-1} 、...、 A_1 、 A_0 组成的地址最小项。

任何一个具有 l 个输入变量的逻辑函数都可以用最小项之和来表示：

$$F = \sum_{i=0}^{2^l-1} m_i (1, \text{or } 0)$$

这里的 m_i 是由函数的输入变量 A 、 B 、 C 、... 组成的最小项。



1) $l \leq n$ 的情况

(l 为函数的输入变量数, n 为选用的MUX的地址输入端数。)

- 当 $l=n$ 时, 只要将函数的输入变量 A 、 B 、 C 、... 依次接到MUX的地址输入端, 根据函数 F 所需要的最小项, 确定MUX中 D_i 的值(0或1)即可;
- 当 $l < n$ 时, 将MUX的高位地址输入端不用(接0或1), 其余同上。

$$Y = \sum_{i=0}^{2^n-1} m_i D_i$$

$$F = \sum_{i=0}^{2^l-1} m_i (1, or 0)$$



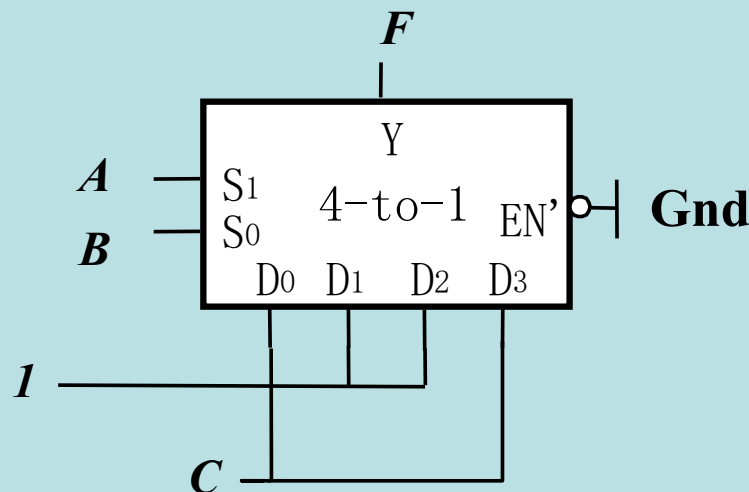
思考：用4选1 MUX实现

$$F = \sum m(1,2,3,4)$$

变量数 $l >$ 数据选择器地址输入数

$$Y = \sum_{i=0}^3 m_i D_i = (A_1 A_0)(D_0 D_1 D_2 D_3)$$

$$\begin{aligned} F &= \sum m(1,2,3,4,5,7) \\ &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + ABC \end{aligned}$$



选2个变量作为 4选1 MUX的两个地址输入。令 $AB=A_1A_0$:

$$\begin{aligned} F &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + ABC \\ &= m_0C + m_1\bar{C} + m_1C + m_2\bar{C} + m_2C + m_3C \\ &= m_0C + m_1 \cdot 1 + m_2 \cdot 1 + m_3C \end{aligned}$$



2) $l > n$ 的情况

如果从 l 个输入变量中选择 n 个直接作为MUX的地址输入，那么，多余的 $(l-n)$ 个变量就要反映到MUX的数据输入 D_i 端，即 D_i 是多余输入变量的函数，简称余函数。

因此设计的关键是如何求出函数 $D_i = f(l-n)$ 。



例：试用4选1MUX实现三变量函数：

$$F = \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$

解：① 首先选择地址输入，令 $A_1A_0=AB$ ，则多余输入变量为 C ，余函数 $D_i=f(c)$ 。

② 确定余函数 D_i 。

用代数法将 F 的表达式变换为与 Y 相应的形式：

$$\begin{aligned} Y &= \overline{A_1}\overline{A_0}D_0 + \overline{A_1}A_0D_1 + A_1\overline{A_0}D_2 + A_1A_0D_3 \\ F &= (\overline{A}\overline{B})C + (\overline{A}\overline{B})\overline{C} + (\overline{A}B)C + (A\overline{B})\overline{C} \\ &= (\overline{A}\overline{B})(C + \overline{C}) + (\overline{A}B)C + (A\overline{B})\overline{C} \\ &= \overline{A}\overline{B} \cdot 1 + \overline{A}B \cdot C + A\overline{B} \cdot \overline{C} + AB \cdot 0 \end{aligned}$$

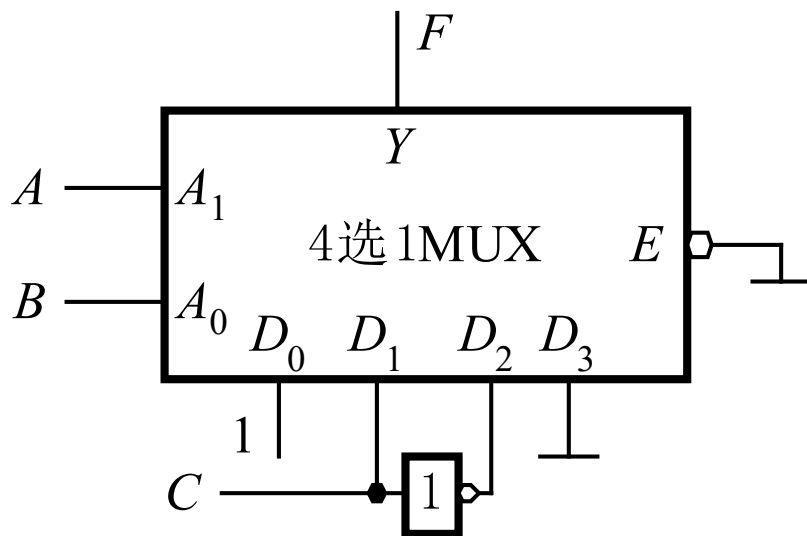
$$Y = \overline{A_1} \overline{A_0} D_0 + \overline{A_1} A_0 D_1 + A_1 \overline{A_0} D_2 + A_1 A_0 D_3$$

$$F = (\overline{A}\overline{B})C + (\overline{A}\overline{B})\overline{C} + (\overline{A}B)C + (A\overline{B})\overline{C}$$

$$= (\overline{A}\overline{B})(C + \overline{C}) + (\overline{A}B)C + (A\overline{B})\overline{C}$$

$$= \overline{A}\overline{B} \cdot 1 + \overline{A}B \cdot C + A\overline{B} \cdot \overline{C} + AB \cdot 0$$

将 F 与 Y 对照可得: $D_0 = 1, D_1 = C, D_2 = \overline{C}, D_3 = 0$





*降维K图法求余子式 D_i

n 变量的逻辑函数，可以用 n 维(即 n 变量)K图表示，也可以用 $(n-1)$ 、 $(n-2)$ 、...维K图表示，这种 $(n-1)$ 、 $(n-2)$ 、...维K图称为降维K图。



*降维K图法求余子式 D_i

$$F = \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$

$\begin{array}{c} AB \\ \swarrow \downarrow \end{array}$ C	00	01	11	10
0	1	0	0	1
1	1	1	0	0

按AB变量组
合重画K-图

$\begin{array}{c} A \\ \swarrow \downarrow \end{array}$ B	0	1
0	1	$\overline{C}$
1	C	0

$\begin{array}{c} A_1 \\ \swarrow \downarrow \end{array}$ A_0	0	1
0	D_0	D_2
1	D_1	D_3

Y

选择地址输入,
令 $A_1A_0=AB$ 。

$\begin{array}{c} AB \\ \swarrow \downarrow \end{array}$ C	00	01	11	10
0	1	0	0	1
1	1	1	0	0

$D_0 \quad D_1 \quad D_3 \quad D_2$

$$D_0 = 1, D_1 = C, D_2 = \overline{C}, D_3 = 0$$



降维K图法求余子式 D_i 的求解步骤归纳如下:

- ① 画出函数 F 的K图。
- ② 选择地址输入。
- ③ 在 F 的K图上确定余函数 D_i 的范围(子K图)。
- ④ 求余函数 D_i 。
- ⑤ 画出逻辑图。



【例4-8】试用8选1MUX实现逻辑函数:

$$F(A,B,C,D) = \sum m(0,4,5,7,12,13,14)$$

① 画出F的四变量K图。

$\begin{array}{c} AB \\ \swarrow \searrow \end{array}$		00	01	11	10
CD	00	1	1	1	0
01	0	1	1	1	
11	0	1	0	0	
10	0	0	1	0	



② 选择地址变量，确定余函数 D_i 。

Method 1:

AB CD	00	01	11	10
00	1	1	1	0
01	0	1	1	1
11	0	1	0	0
10	0	0	1	0

选择地址 ($A_2A_1A_0=ABC$)

降维K图:

AB C	00	01	11	10
0	$\overline{D}$	1	1	D
1	0	D	$\overline{D}$	0

F

A_2A_1 A_0	00	01	11	10
0	D_0	D_2	D_6	D_4
1	D_1	D_3	D_7	D_5

F

分别确定 D_0 、 D_1 、...、 D_7 。



Method 2:

AB		00	01	11	10
CD	00	1 D_0 1		1 D_4 0	
	01	0 D_1 1		1 D_5 1	
	11	0 D_3 1		0 D_7 0	
	10	0 D_2 0		1 D_6 0	

选择地址 ($A_2A_1A_0=ACD$)

降维K图:

AC		00	01	11	10
D	0	1	0	B	B
	1	B	B	0	1

F

A_2A_1		00	01	11	10
A_0	0	D_0	D_2	D_6	D_4
	1	D_1	D_3	D_7	D_5

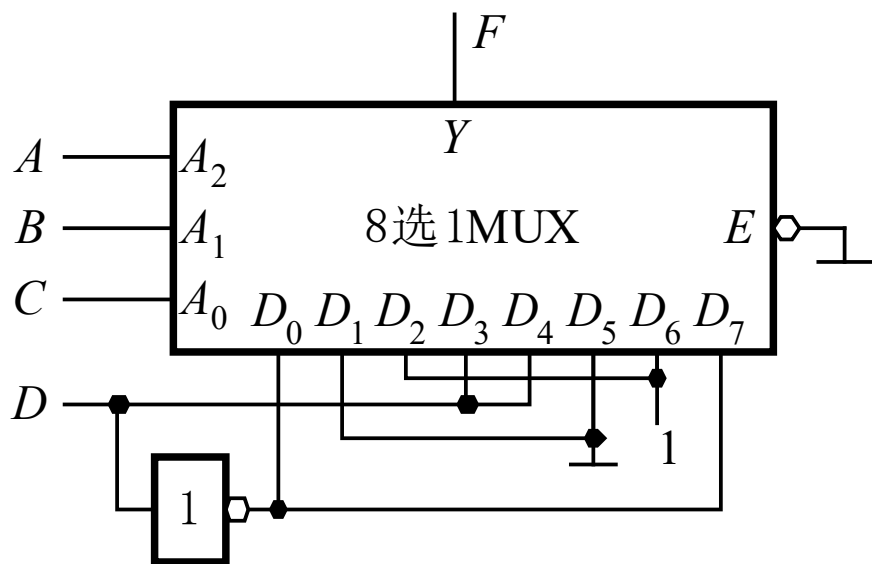
F

分别确定 D_0 、 D_1 、...、 D_7 。



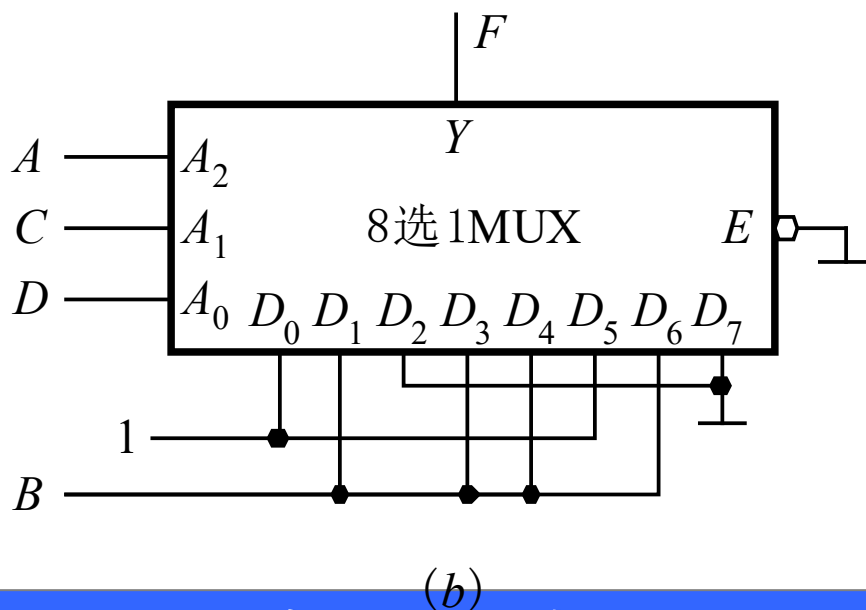
C \ AB				
	00	01	11	10
0	$\overline{D}$	1	1	D
1	0	D	$\overline{D}$	0

F



D \ AC				
	00	01	11	10
0	1	0	B	B
1	B	B	0	1

F



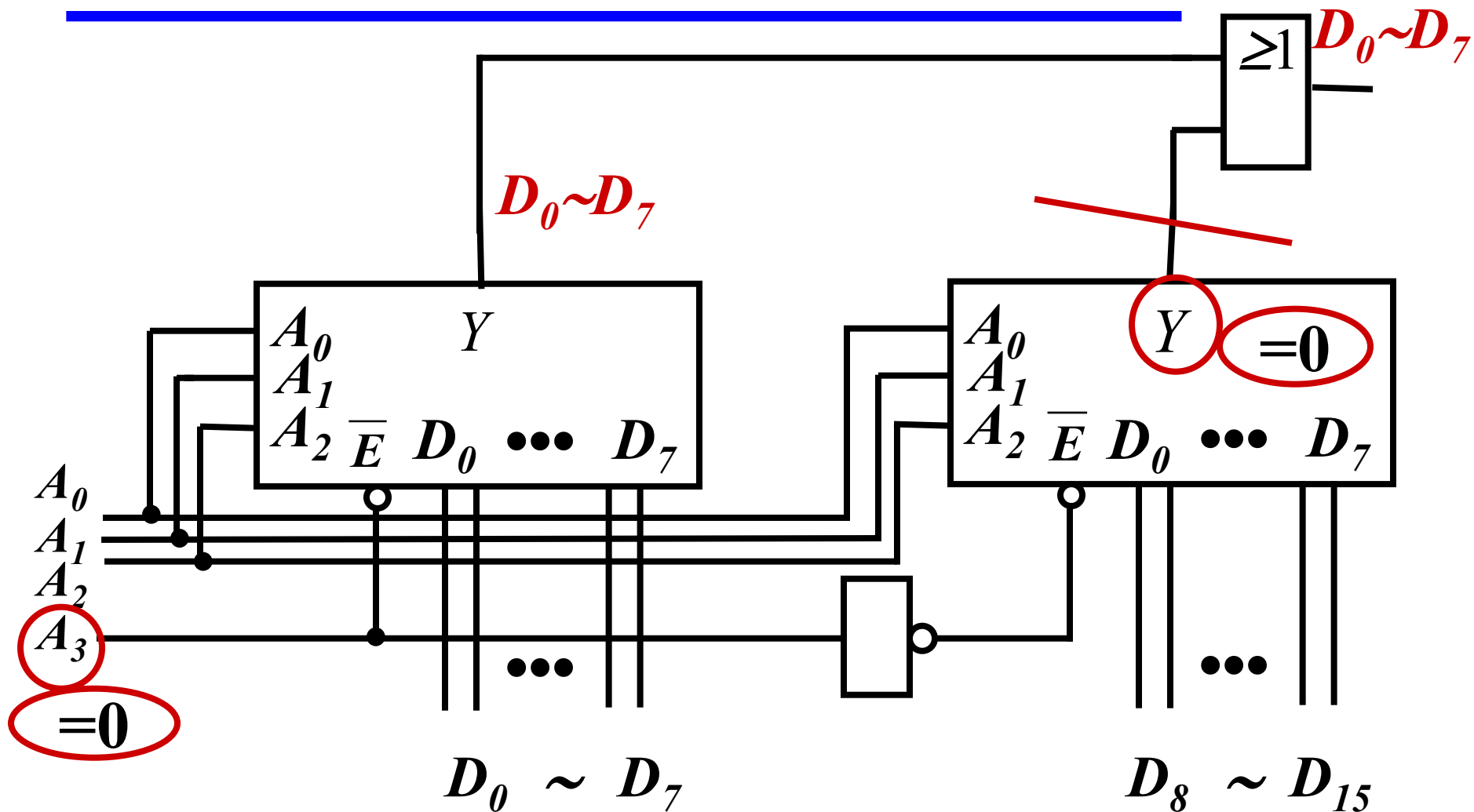


扩展1:

数据选择器的扩展应用

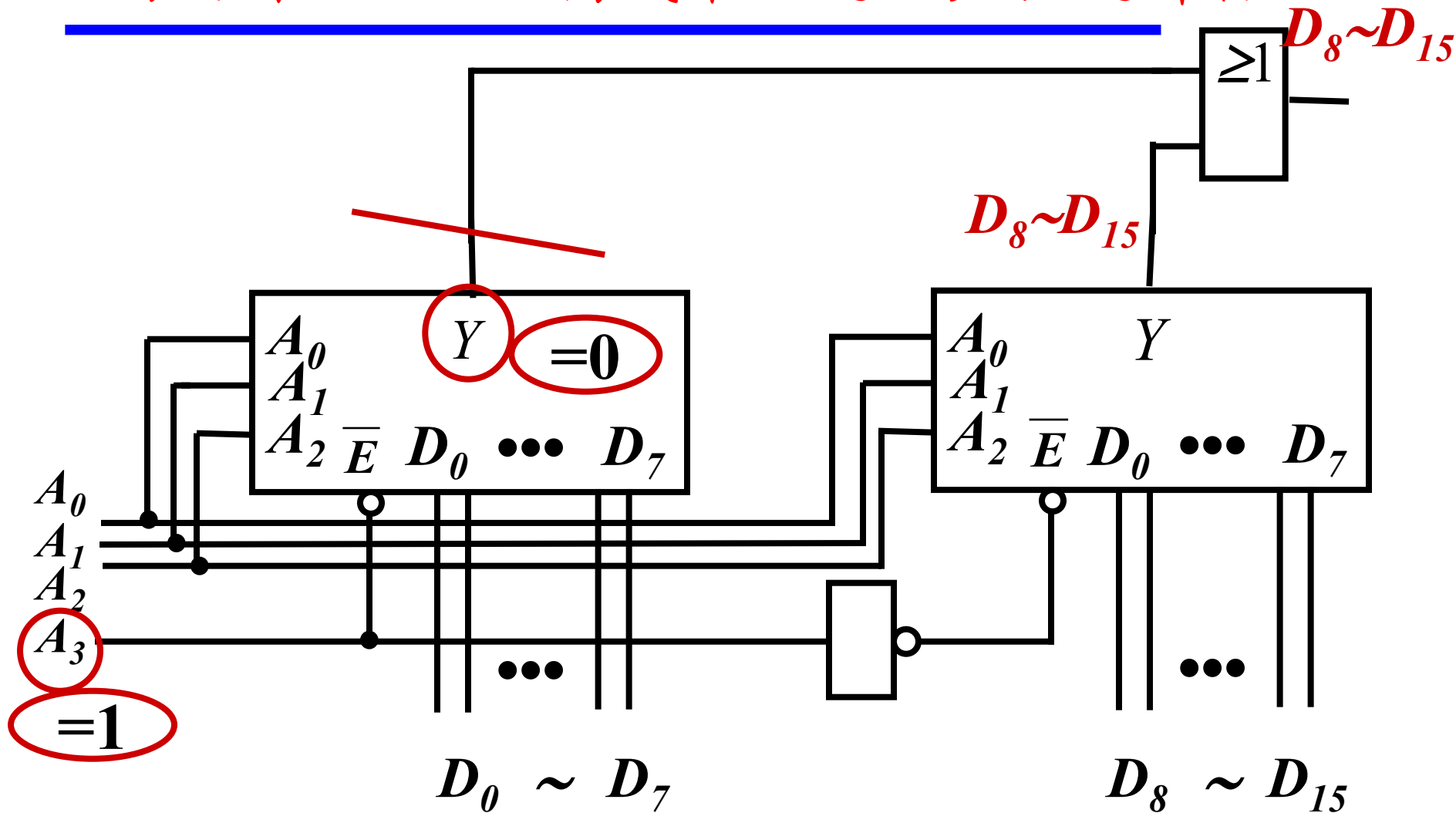


用两片74LS151构成十六选一数据选择器





用两片74LS151构成十六选一数据选择器





*三、数码比较器

比较两个数的大小或是否相等。

1) 一位比较器

2) 四位比较器



* (1) 一位数值比较器

功能表

输入		输出		
A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0



*

输入		输出		
A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

$$“A > B” = \overline{A}B$$

$$“A = B” = \overline{A}\overline{B} + AB$$

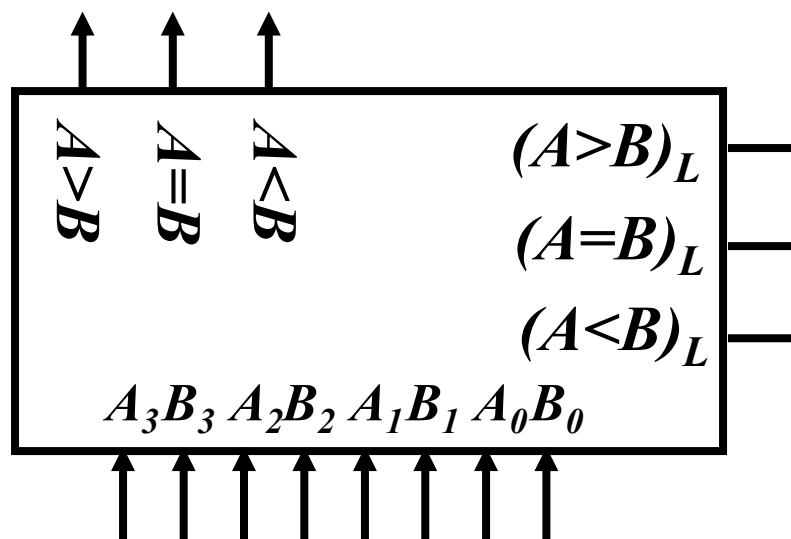
$$“A < B” = A\overline{B}$$



(2) 四位数值比较器

比较原则:

- A. 先从高位比起,高位大的数值一定大。
- B. 若高位相等,则再比较低位数,最终结果由低位的比较结果决定。



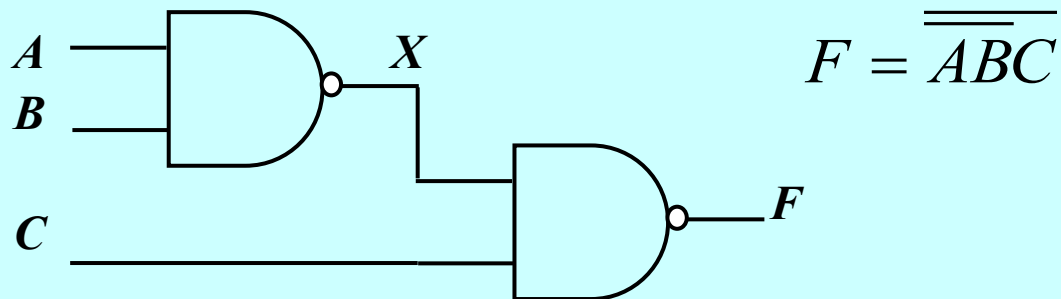


4.4 组合逻辑电路中的竞争与冒险

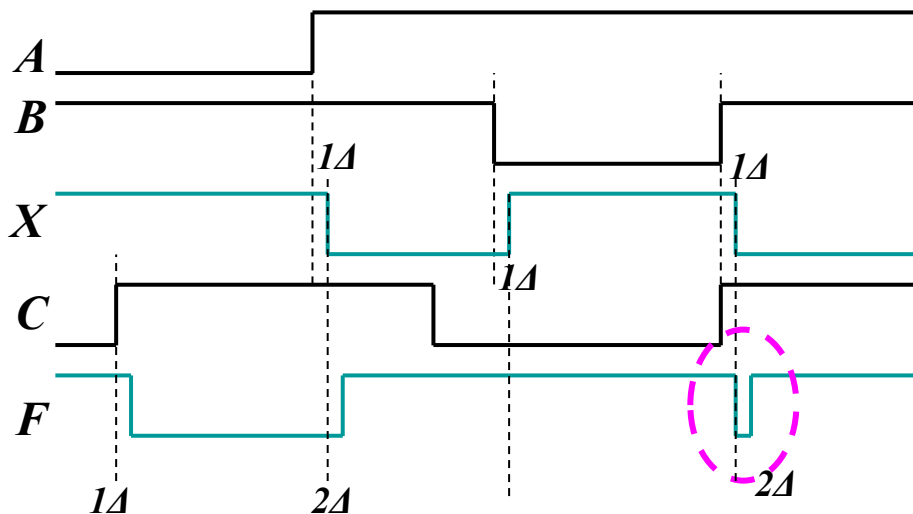
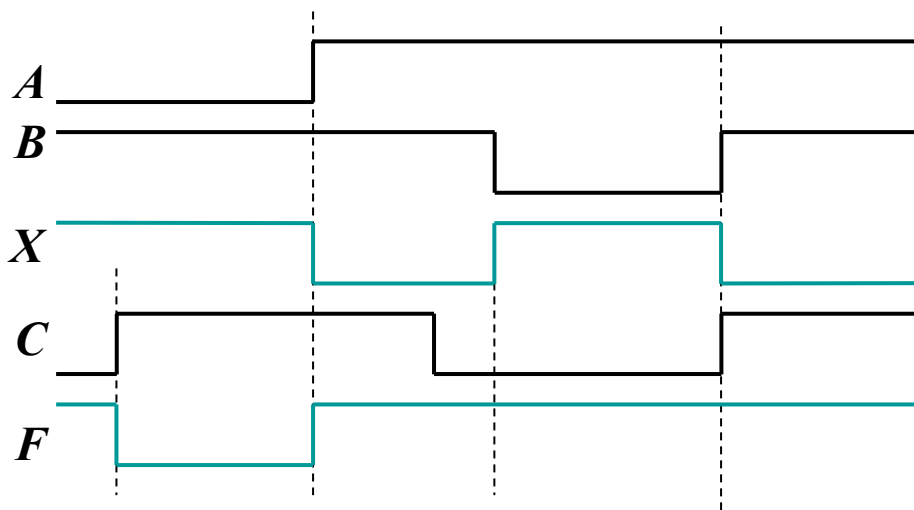
1. 竞争与冒险

在组合电路中，某一输入变量经不同途径传输后，到达电路中某一会合点的时间有先有后，这种现象称为**竞争**。由于竞争而使电路输出发生瞬时错误的现象称为**冒险**。

例:



不考虑器件延时, Δ .



考虑器件延时, Δ .

“冒险” 或 “毛刺”



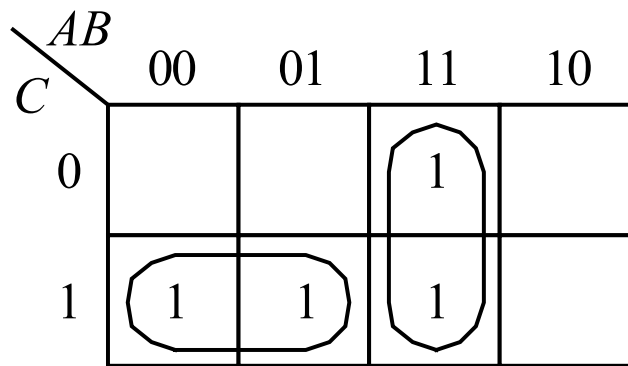
注意：竞争是经常发生的，但
不一定都会产生毛刺。



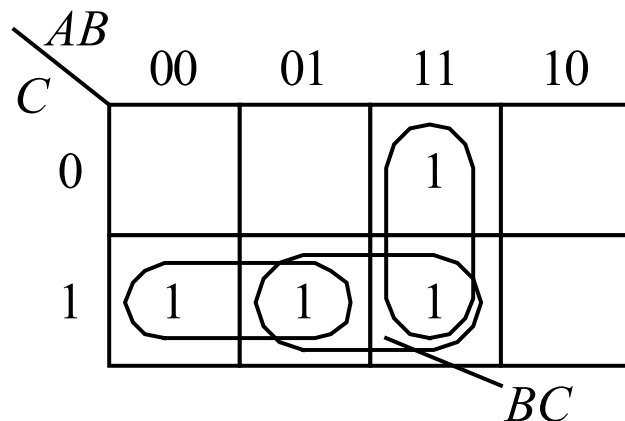
2. 竞争与冒险的识别

① **代数法**。当函数表达式在一定条件下可以简化成 $F=X+\overline{X}$, 或 $F=X\cdot\overline{X}$ 的形式时, X 的变化可能引起冒险现象。

② **K图法**。如果两卡诺圈相切, 而相切处又未被其它卡诺圈包围, 则可能发生冒险现象。



(a)

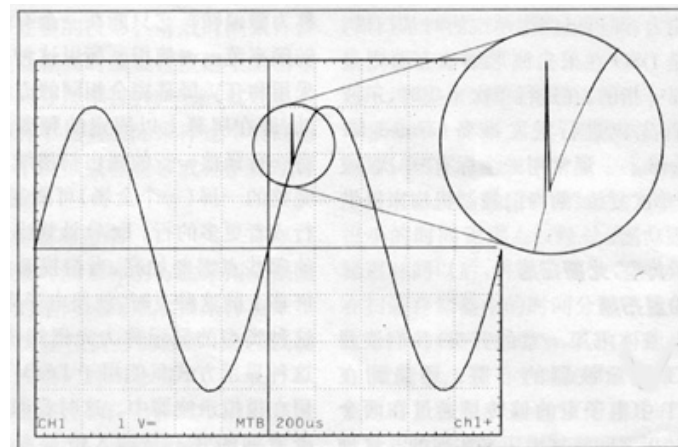
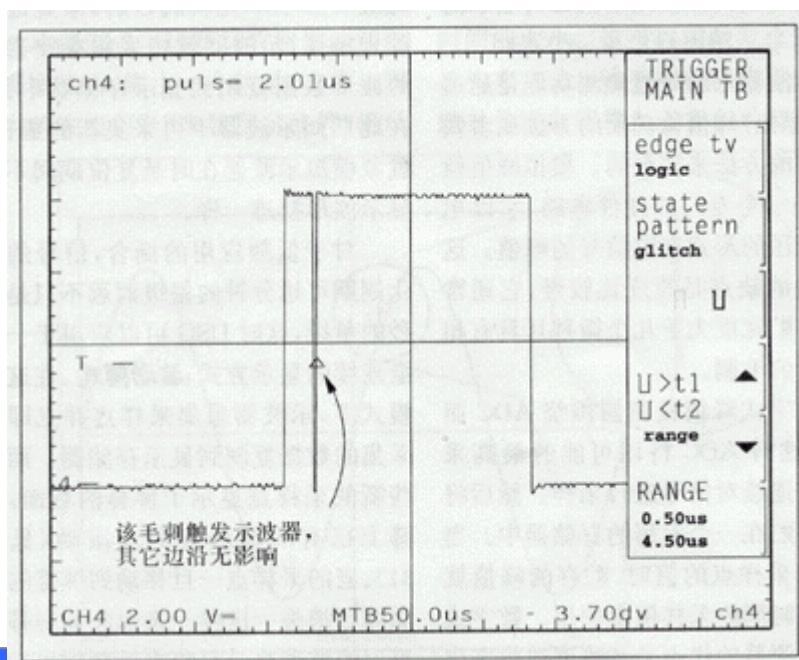


(b)



③ 实验法。两个以上的输入变量同时变化引起的功能冒险难以用上述方法判断。因而发现冒险现象最有效的方法是实验。

利用示波器仔细观察在输入信号各种变化情况下的输出信号，发现毛刺则分析原因并加以消除，这是经常采用的办法。





3. 冒险现象的消除

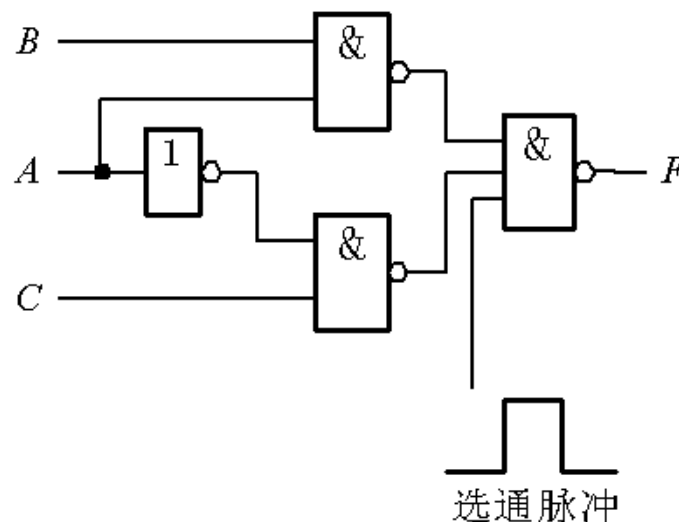
当电路中存在冒险现象时，必须设法消除它，否则会导致错误结果。消除冒险现象通常有如下方法：

① 增加冗余项消除逻辑冒险。

② 加选通信号，避开毛刺。

毛刺仅发生在输入信号变化的瞬间，因此在这段时间内先将门封住，待电路进入稳态后，再加选通脉冲选取输出结果。

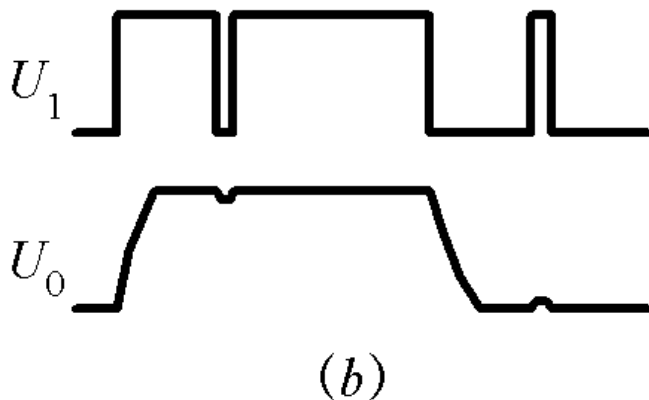
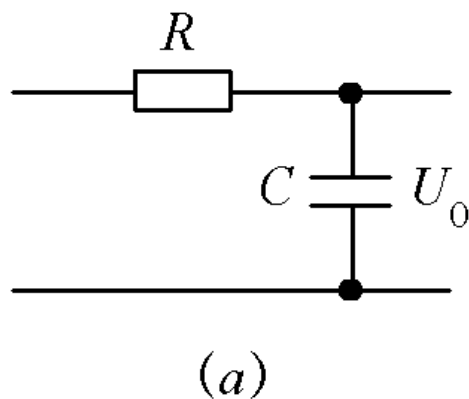
该方法简单易行，但选通信号的作用时间和极性等一定要合适。





③ 加滤波电路，消除毛刺的影响。（务必谨慎）

毛刺很窄，其宽度可以和门的传输时间相比拟，因此常在输出端并联滤波电容 C ，或在本级输出端与下级输入端之间，串接一个积分电路来消除其影响。但 C 或 R 的引入会使输出波形边沿变斜，故参数要选择合适的，一般由实验确定。



加滤波电路排除冒险



总结:

以上三种方法各有特点。

增加冗余项适用范围有限;

加滤波电容是实验调试阶段常采取的应急措施;

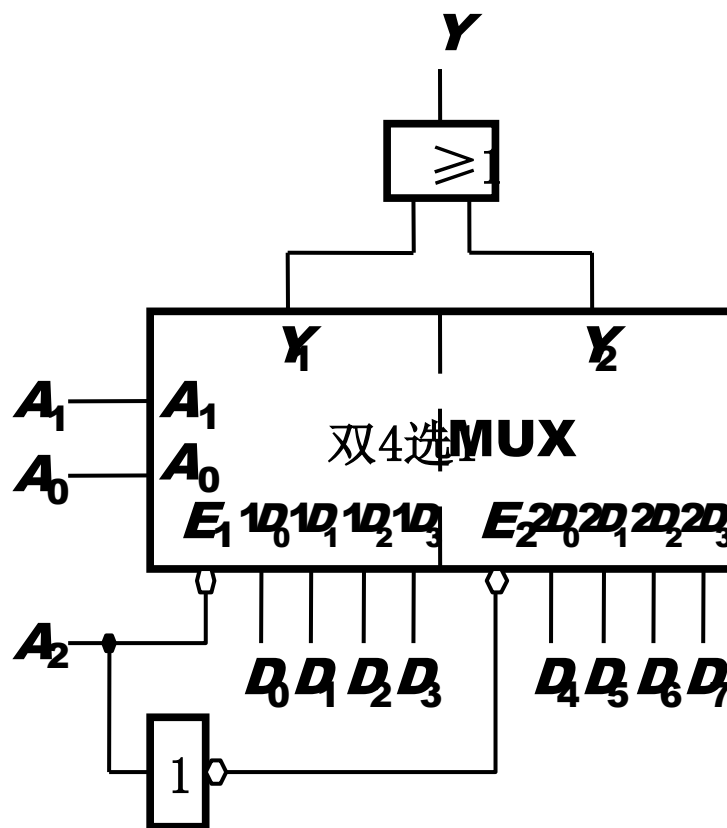
加选通脉冲则是行之有效的方法。目前许多MSI器件都备有使能(选通控制)端, 为加选通信号消除毛刺提供了方便。



练习：

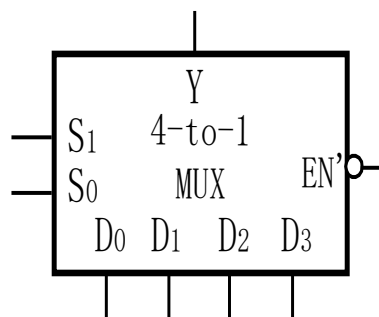
例.试用两个4选1 MUX扩展为8选1 MUX。

A_2	A_1	A_0	Y
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7





用4-to-1MUX实现16-to-1MUX



A_3	A_2	A_1	A_0	Y
0	0	0	0	D_0
0	0	0	1	D_1
0	0	1	0	D_2
0	0	1	1	D_3
0	1	0	0	D_4
0	1	0	1	D_5
0	1	1	0	D_6
0	1	1	1	D_7
1	0	0	0	D_8
1	0	0	1	D_9
1	0	1	0	D_{10}
1	0	1	1	D_{11}
1	1	0	0	D_{12}
1	1	0	1	D_{13}
1	1	1	0	D_{14}
1	1	1	1	D_{15}

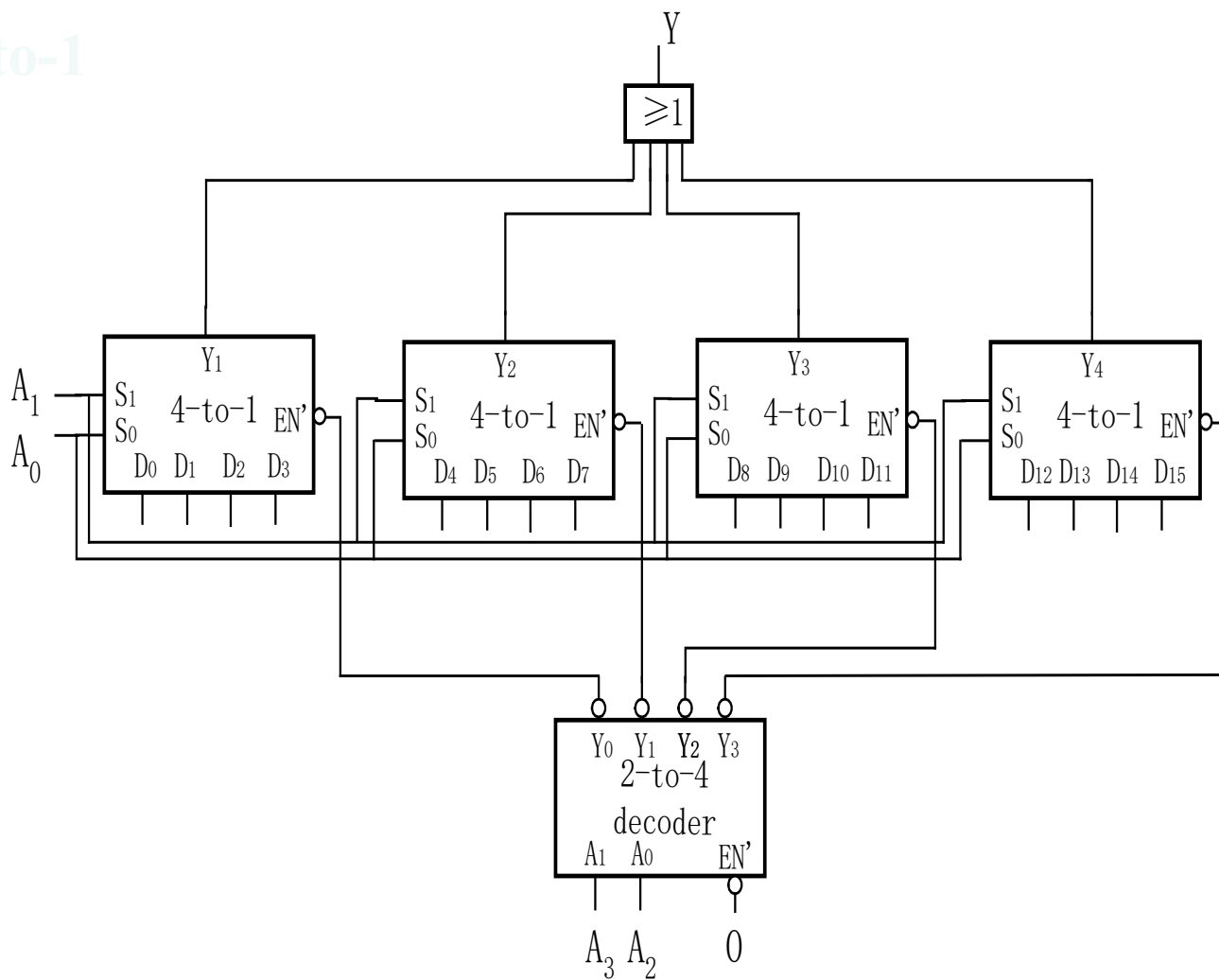
实现方案:

1. 用 A_3 和 A_2 控制4-to-1的 使能
2. 用 A_3 和 A_2 控制4-to-1的 输出



方案一：用 A_3 和 A_2 控制4-to-1的 使能实

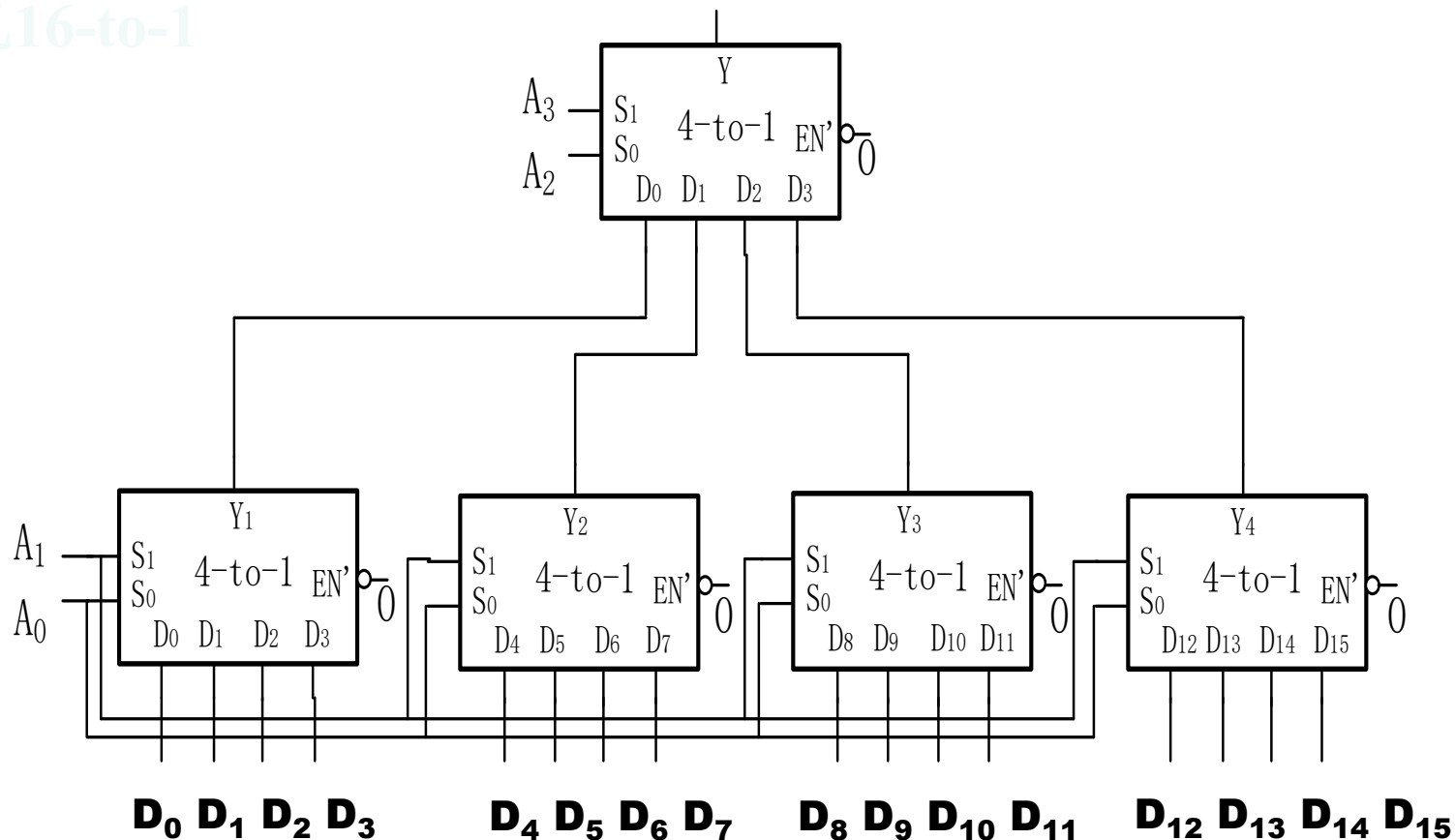
现16-to-1





方案二：用 A_3 和 A_2 控制4-to-1的输出

实现16-to-1



本章小结

- 组合电路的分析
- 组合电路的设计
- 中规模组合集成组合电路
 - ☆ 译码器
 - ☆ 编码器
 - ☆ 数据选择器 (MUX)
 - ☆ 数据分配器
 - ☆ 数码比较器
 - ☆ 加法器减法器
- 组合电路中的竞争与冒险现象

本章作业

- P76

3-2, 3-4, 3-5, 3-6, 3-8, 3-11,
3-15(1),(2), 3-19, 3-20, 3-22